Steer Unified Audio Editing Toward Reliable Source Selection

Youquan Fu[1,2,*], **Dongnuan Cai**[1], **Yu Pan**[1], **Henghui Du**[2], **Yujia Zhang**[2], **Ping Luo**[3], **Quande Liu**[4], **Yake Wei**[1,†], **Xi Chen**[2,†], **Hongyuan Zhang**[3,†], **Di Hu**[1]

[1]Renmin University of China, [2]Tencent, [3]The University of Hong Kong, [4]The Chinese University of Hong Kong

[*]Work done during an internship at Tencent. [†]Corresponding author.

Abstract

Unified audio editors support diverse operations, yet often struggle to modify the intended source while preserving non-target content. We show that source selection in audio editing is shaped by early denoising states, while incorrect early selections can hardly be corrected during subsequent denoising. Motivated by these findings, we propose *Source-selection Training via Early-trajectory Emphasis and Recovery (STEER)*, a two-stage post-training algorithm for reliable selective audio editing. STEER-Emphasis prioritizes training on task-critical early states to reduce source-selection errors, while STEER-Recovery learns corrective denoising from self-generated rollout states to recover from early deviations. To systematically assess source-selection reliability, we introduce the Selective Audio Editing Benchmark, enabling large-scale automatic evaluation of target manipulation and non-target preservation. Applying STEER to our pretrained multitask base editor gives SteerSound, a unified audio editor with leading selective editing performance among open-source audio editors. Experiments on our base editor, UNISON, and SAM Audio demonstrate that STEER consistently improves both selective editing capability and overall editing performance. Early-trajectory emphasis further brings substantial improvements in transcript-conditioned target speaker removal and separation, highlighting the broader applicability of this training principle.

Project Page: https://gewu-lab.github.io/SteerSound/
Code: https://github.com/GeWu-Lab/SteerSound
Checkpoint: https://huggingface.co/GeWu-Lab/SteerSound
Dataset: https://huggingface.co/datasets/GeWu-Lab/SteerSound

1 Introduction

Recent advances in diffusion and Flow Matching have enabled unified audio editors to follow diverse instructions and perform multiple editing operations within a single model [16, 30–32]. However, reliable unified editing requires more than broad task coverage. Editing tasks like removal, replacement, and pitch or volume modification require the model to *identify the intended source*, apply the requested operation, and preserve the remaining content. We refer to these as selective editing tasks. As illustrated in Figure 1 (left), even strong existing models, like UNISON [16] and SAM Audio [28], can leave the intended source unchanged, modify a non-target source, or alter both. These selective editing failures highlight the need to strengthen not only audio generation, but also the model's ability to select the correct editing source.

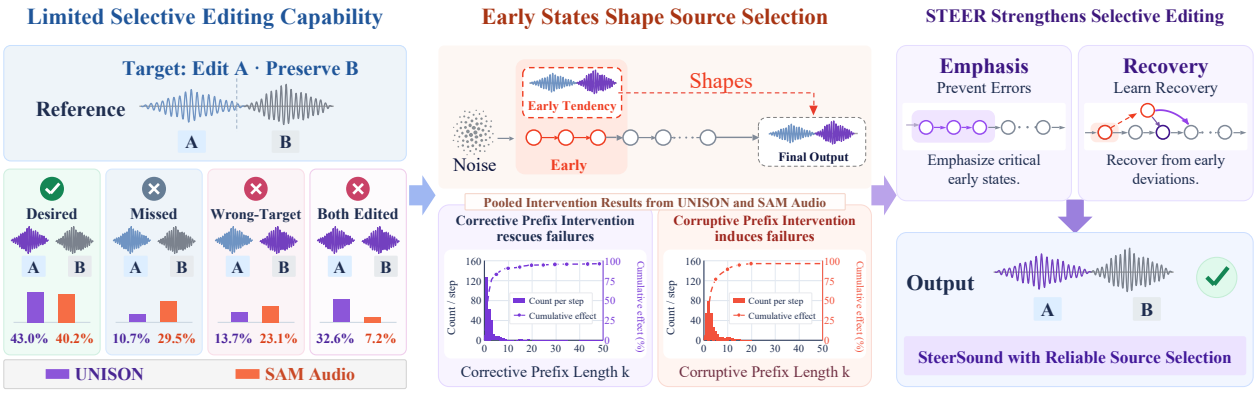


Figure 1 Strengthening selective audio editing with STEER. **Left**: Existing editors may leave the target unchanged or modify non-target content. **Middle**: Early states shape final source selection, while later denoising tends to follow the established editing direction rather than revise it. **Right**: STEER addresses these failures through early-trajectory emphasis (Emphasis) and recovery training (Recovery), yielding SteerSound with more reliable selective editing.

To understand these failures, we investigate how source selection develops along the generation trajectory. By analyzing the editing tendency at different denoising steps, we find that incorrect source-selection tendencies form early in the generation trajectory. This suggests that establishing the correct source selection early may be critical to the final editing outcome. As shown in Figure 1 (middle), we conduct bidirectional early-trajectory interventions: replacing the early trajectories of failed edits with correct ones, and those of successful edits with deliberately incorrect ones. Across UNISON and SAM Audio, correct early trajectories can rescue failed edits, whereas incorrect ones can induce source-selection failures, showing that early source selection strongly shapes which source is ultimately edited.

Together, these experiments reveal a common pattern across selective editing tasks: the source ultimately edited is largely determined by the early generation trajectory. Once an incorrect source is selected early in denoising, later denoising tends to follow the resulting trajectory rather than revise the earlier decision. These findings motivate us to address these source-selection failures from two complementary directions: preventing source-selection errors from forming early and improving recovery when they occur. We therefore propose *Source-selection Training via Early-trajectory Emphasis and Recovery (STEER)*, a two-stage post-training algorithm that strengthens source selection in pretrained audio editors. In the first stage, STEER-Emphasis places greater training emphasis on critical early-trajectory states to reduce the formation of incorrect source selections. In the second stage, STEER-Recovery learns corrective denoising from self-generated rollout states to recover from the early deviations that remain, as illustrated in Figure 1 (right).

To enable more comprehensive and systematic evaluation, we first introduce the Selective Audio Editing Benchmark for large-scale assessment of source-selection reliability. Moreover, existing unified audio editors remain limited in selective task coverage, making broad analysis across diverse editing operations difficult. We therefore develop our base editor, an extensively pretrained unified editor covering six generation and editing tasks, including four selective editing tasks, providing a stronger and more comprehensive foundation for selective audio editing and cross-task analysis. Applying STEER to our base editor gives SteerSound, which substantially improves source-selection reliability while achieving advanced editing performance among open-source unified audio editors. Beyond SteerSound, STEER can be well applied to UNISON and SAM Audio, enhancing their selective editing performance, while early-trajectory emphasis further applies to transcript-conditioned target speaker removal and separation, supporting the broader applicability of our approach.

Our contributions are summarized as follows. **Firstly**, through intermediate-state analysis and bidirectional prefix interventions, we show that early denoising states largely determine whether a unified audio editor edits the correct source, and hence its selective editing performance. **Secondly**, we propose STEER, a two-stage training algorithm: STEER-Emphasis prioritizes critical early-trajectory states, while STEER-Recovery learns corrective denoising from self-generated rollout states. STEER improves selective and overall editing performance on our base editor, UNISON, and SAM Audio, and the early-trajectory emphasis further extends

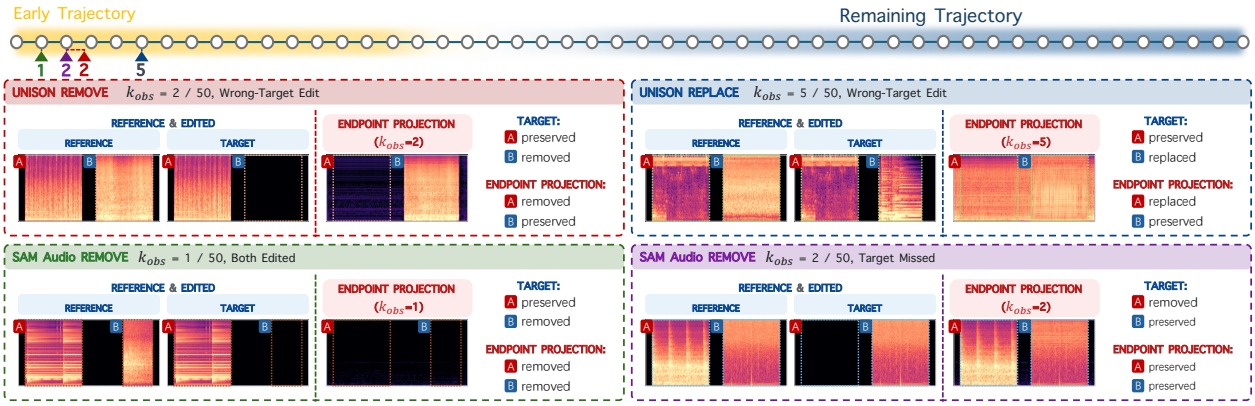


Figure 2 Erroneous source selection is formed and visible within the first few denoising steps. Each panel shows a failed edit: the reference, the target, and the zero-noise estimate decoded at step k_{obs}, the earliest step at which the erroneous tendency becomes perceptible. A and B denote the two sources in the reference audio.

to related selective-control tasks. **Finally**, we introduce the Selective Audio Editing Benchmark, enabling large-scale automatic evaluation of editing success and three types of selective failures across six tasks. We also develop a six-task unified base editor and apply STEER to obtain SteerSound, which achieves leading overall and selective editing performance among open-source audio editors.

2 Related Work

Unified and Instruction-Guided Audio Editing. Text-guided audio editing has been explored through training-based and training-free diffusion approaches, including AUDIT [32], ZETA [19], PPAE [33], and AudioEditor [11], while recent systems such as MMEdit [30], Audio-Omni [31], and UNISON [16] further broaden task coverage and integrate audio-language modeling with audio editing. However, source-selection reliability in unified audio editors remains underexplored. We develop SteerSound, a unified six-task audio editor, and strengthen this selective editing capability.

Denoising Dynamics for Source Selection. Prior work has shown that denoising timesteps can play non-uniform roles, through stage-specialized denoisers [1], timestep reweighting or sampling [6, 8], and task-dependent timestep emphasis in diffusion-based visual perception [22]. Related studies also use model-generated states to reduce train–inference mismatch or trajectory errors, such as Self Forcing [10] for autoregressive video diffusion and SOAR [24] for diffusion post-training. However, these works do not study how an audio editor selects which source to modify, nor how such source-selection decisions evolve along the denoising trajectory. We show that source selection is established along a task-dependent early trajectory, motivating greater training emphasis on critical early states and recovery from early deviations.

3 Source Selection Along the Trajectory

To strengthen source selection in unified audio editing, we examine how source-selection decisions evolve during generation. We first investigate when source-selection tendencies emerge and then test the role of early-trajectory states in final source selection on two representative editors: UNISON [16], a representative unified audio editor, and SAM Audio [28], a representative source separation model that we evaluate for source removal.

Flow Matching and trajectory notation. Given a clean target latent $\mathbf{z}_0$ and Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, Flow Matching [17] constructs $\mathbf{z}_\sigma = (1-\sigma)\mathbf{z}_0 + \sigma\boldsymbol{\epsilon}$ and optimizes

$$\mathcal{L}_{\text{FM}} = \|f_\theta(\mathbf{z}_\sigma, \sigma, c) - (\boldsymbol{\epsilon} - \mathbf{z}_0)\|_2^2, \tag{1}$$

where c denotes the editing condition. We denote the denoising schedule by $1 = \sigma_0 > \sigma_1 > \cdots > \sigma_K = 0$ and the state after k steps by $\mathbf{z}^{(k)}$, with $\mathbf{z}^{(k)} = \mathbf{z}^{(k-1)} + (\sigma_k - \sigma_{k-1})\mathbf{v}_{k-1}$. Unless otherwise specified, we use

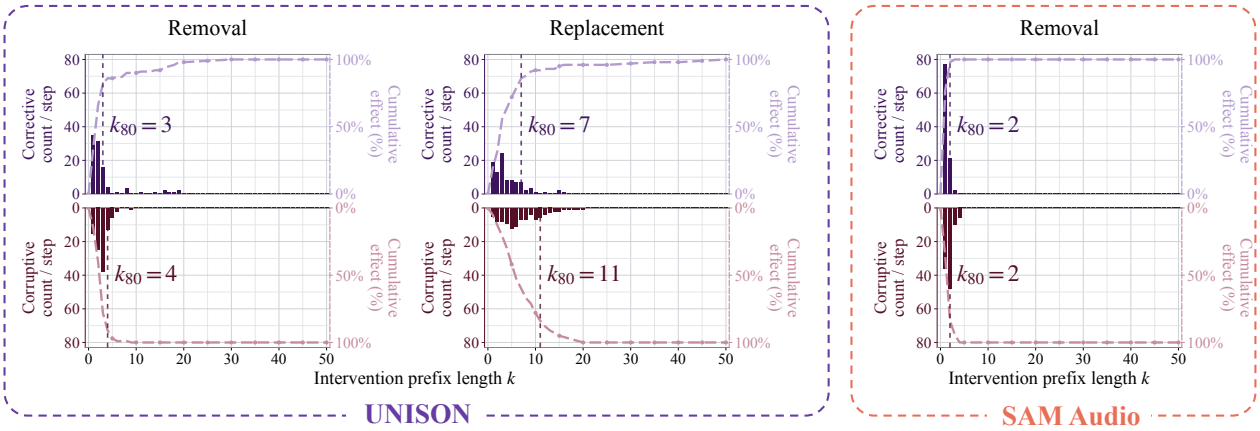


Figure 3 Bidirectional early-trajectory interventions on source selection. Corrective (top) and corruptive (bottom) prefix interventions are evaluated on 100 validation examples per task. Bars show per-step corrective or corruptive counts, while curves show the corresponding cumulative effects. For each task, k_{80} denotes the shortest intervention prefix length at which the cumulative effect reaches 80%. Removal consistently exhibits a shorter k_{80} than replacement.

$K = 50$ denoising steps.

3.1 Emergence of Source-Selection Tendencies

To examine when source-selection tendencies emerge, we inspect the editing direction represented at intermediate stages of the model's denoising trajectory. Since intermediate states remain noisy, we estimate their editing direction by extrapolating each state to the zero-noise endpoint using its instantaneous velocity prediction:

$$\hat{\mathbf{z}}_0^{(k)} = \mathbf{z}^{(k)} - \sigma_k \mathbf{v}_k, \tag{2}$$

where $\mathbf{v}_k = f_\theta(\mathbf{z}^{(k)}, \sigma_k, c)$. Decoding $\hat{\mathbf{z}}_0^{(k)}$ reveals the source-selection tendency at step k, allowing us to inspect which source the model tends to edit and which it tends to preserve. This allows us to identify when an erroneous source-selection tendency begins to form during generation. We apply this probe to failed examples from selective editing tasks across the two models. We observe that erroneous source-selection tendencies are already formed and perceptually discernible during early denoising, as illustrated by the representative cases in Figure 2. In these examples, UNISON edits the wrong source, while SAM Audio either removes both sources or misses the target. Notably, the zero-noise estimates reveal these errors within the first 1–5 of 50 denoising steps, even though the underlying states remain noisy.

3.2 Early Trajectories Shape Final Source Selection

To test whether early trajectory states causally influence final source selection, we perform corrective and corruptive prefix interventions with varying lengths k. For failed edits, we replace the first k model-predicted velocities with oracle velocities constructed from the correct editing target. For successful edits, we instead replace the first k velocities with those constructed from a deliberately corrupted target encoding incorrect source selection. In both cases, the model resumes its own denoising under the original instruction.

As shown in Figure 3, short corrective prefixes can rescue failed edits, whereas short corruptive prefixes can induce source-selection failures. Across the evaluated tasks and models, extending these prefixes into later denoising stages yields limited additional effects. These results provide evidence that early trajectory states shape final source selection, while subsequent denoising often follows the editing direction established early, even when it is incorrect. Two properties of standard training may partly explain this behavior. SD3-style logit-normal sampling [6] emphasizes intermediate noise levels, leaving high-noise early states relatively underrepresented. Meanwhile, training conditions on noisy versions of correct targets, whereas inference states are self-generated, which may make recovery from early source-selection errors difficult.

Different editing tasks exhibit different sensitivities to early prefix replacement, reflecting differences among editing operations. Removal sharply contrasts target suppression with non-target preservation, making source selection apparent within only a few steps, whereas replacement requires more steps for the modification to

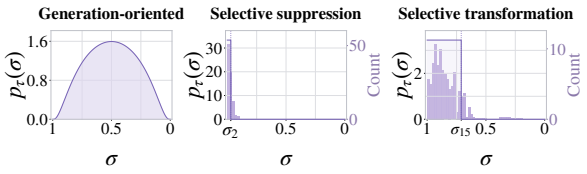


Figure 4 Noise-level sampling distributions for different task groups in STEER-Emphasis.

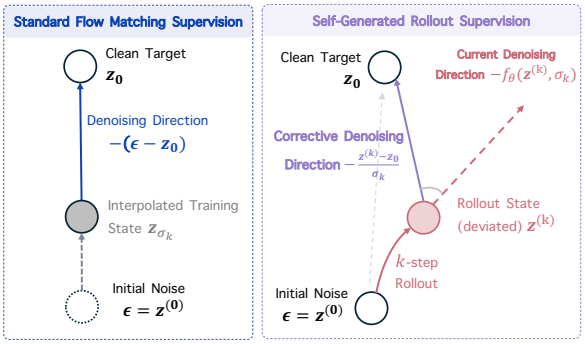


Figure 5 Comparison of standard Flow Matching supervision on interpolated states and STEER-Recovery supervision on self-generated rollout states.

Algorithm 1: STEER

Input: θ_0, $\mathcal{D}_{\text{tr}}$, $\mathcal{D}_{\text{val}}$, p_τ, K, P
Output: $\theta^\star$

Stage I: STEER-Emphasis
Train θ_0 with $\mathcal{L}_{\text{FM}}$ and $\sigma \sim p_\tau$
$\theta^\star \leftarrow$ validation-best checkpoint
Stage II: STEER-Recovery
$k \leftarrow 1,\ q \leftarrow 0$
while $q < P$ **and** $k < K$ **do**
  $\theta \leftarrow \theta^\star$; reset optimizer and schedule
  for each training update **do**
    Sample $(\mathbf{z}_0, c, \tau) \sim \mathcal{D}_{\text{tr}}$
    Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
    if $\tau \in$ {Generate, Add} **then**
      $\sigma \sim p_{\text{LN}}$; $\mathcal{L} \leftarrow \mathcal{L}_{\text{FM}}$
    else
      $\mathbf{z}^{(k)} \leftarrow \text{sg}(\textsc{Rollout}(\theta, \boldsymbol{\epsilon}, c, k))$
      $\mathcal{L} \leftarrow \mathcal{L}_{\text{rec}}^{(k)}$
    end if
    Update θ using $\mathcal{L}$
  end for
  $\hat{\theta}_k \leftarrow$ validation-best checkpoint at depth k
  if $S_{\text{val}}(\hat{\theta}_k) > S_{\text{val}}(\theta^\star)$ **then**
    $\theta^\star \leftarrow \hat{\theta}_k,\ q \leftarrow 0$
  else
    $q \leftarrow q + 1$
  end if
  $k \leftarrow k + 1$
end while
return $\theta^\star$

become distinguishable, yielding broader but still early critical ranges. We therefore group operations into *generation-oriented*, *selective suppression*, and *selective transformation* tasks, depending on whether they synthesize new content, apply strongly contrasting operations to the target and preserved sources, or modify the target's content or acoustic attributes.

4 Unified Audio Editing with STEER

Motivated by the findings in Section 3, we aim to develop a unified audio editor that combines broad task coverage with reliable source selection. Section 4.1 introduces our base editor, which supports six audio generation and editing tasks, including four selective editing tasks. Section 4.2 then presents STEER, a two-stage post-training algorithm for improving source-selection reliability.

4.1 Unified Audio Editor

To develop a unified audio editor that combines broad task coverage with strong selective editing capabilities, we first build a multi-task base editor. Unlike existing editors, which typically support only a subset of the selective operations considered here, our base editor supports six tasks: two generation-oriented tasks (generation and addition) and four selective editing tasks (removal, replacement, pitch modification, and volume modification). This unified task coverage also enables systematic cross-task analysis of source selection within a single model. We pretrain the model on large-scale audio editing data to establish strong initial multi-task audio editing capabilities. Further details on our base editor are provided in Section 5.1.

4.2 STEER

STEER enhances source-selection reliability in pretrained audio editors through two complementary post-training stages. First, STEER-Emphasis prioritizes training on task-critical early denoising states to reduce the formation of source-selection errors. STEER-Recovery then learns corrective denoising from self-generated rollout states to recover from remaining early-trajectory deviations.

4.2.1 Stage1: Early-Trajectory Emphasis

Standard logit-normal sampling emphasizes intermediate noise levels, leaving early states critical to source selection relatively underrepresented during training. To address this, STEER-Emphasis prioritizes training

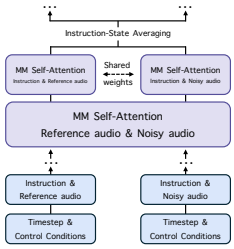


Figure 6 Architecture of our base editor.

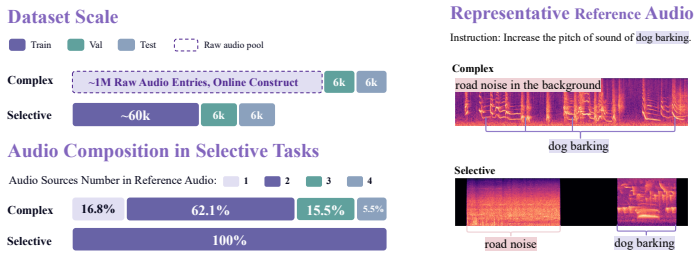


Figure 7 Statistics and representative examples of the two data settings, including dataset scale and source composition.

on critical early denoising states to reduce source-selection errors. Since different editing tasks exhibit different early-trajectory sensitivities, we first identify their critical ranges and then increase sampling within the corresponding intervals.

To identify these ranges, we perform corrective prefix interventions on our base editor. For each failed edit, we determine the shortest early prefix whose correction leads to a successful edit. Let $C_\tau(k)$ denote the cumulative fraction of failed edits for task τ rescued by corrective prefix interventions of up to k denoising steps. Following the task grouping in Section 3.2, we define the critical range for each selective task group $\mathcal{G}$ as $k^\rho_{\mathcal{G}} = \max_{\tau\in\mathcal{G}} \min\{k : C_\tau(k) \geq \rho\}$, where ρ specifies the fraction of rescued edits to be covered. Taking the maximum ensures that the range covers the critical prefix of each task in the group. With $\rho = 0.8$, the resulting ranges cover the first 2 of 50 denoising steps for selective suppression and the first 15 for selective transformation.

Based on these ranges, we uniformly sample noise levels within the corresponding high-noise intervals for selective editing tasks, while retaining logit-normal sampling for generation-oriented tasks:

$$p_\tau(\sigma) = \begin{cases} p_{\mathrm{LN}}(\sigma), & \tau \in \{\text{Generate}, \text{Add}\}, \quad \textit{generation-oriented} \\ \mathcal{U}(\sigma_2, 1), & \tau = \text{Remove}, \quad \textit{selective suppression} \\ \mathcal{U}(\sigma_{15}, 1), & \tau \in \{\text{Replace}, \text{Pitch}, \text{Volume}\}, \quad \textit{selective transformation} \end{cases} \tag{3}$$

Here p_{LN} denotes the SD3-style logit-normal distribution [6], defined by $\sigma = \mathrm{sigmoid}(u)$ with $u \sim \mathcal{N}(0, 1)$. STEER-Emphasis increases sampling of critical early states, while leaving the Flow Matching objective unchanged. Figure 4 illustrates the sampling distributions, with additional intervention results provided in Appendix A.

4.2.2 Stage 2: Early-Trajectory Recovery

Although STEER-Emphasis reduces early source-selection errors, incorrect selections may still occur and persist through subsequent denoising. Standard Flow Matching supervises states interpolated between Gaussian noise and the clean target. Consequently, when early errors drive inference onto incorrect trajectories, subsequent denoising may continue along these trajectories rather than correcting them. To address this, STEER-Recovery introduces corrective supervision at self-generated rollout states, as illustrated in Figure 5, enabling the model to learn to recover from its own trajectory deviations.

Starting from the validation-best STEER-Emphasis checkpoint, we perform a k-step denoising rollout from Gaussian noise $\boldsymbol{\epsilon}$ to obtain $\mathbf{z}^{(k)}$ for each selective editing example, without backpropagating through the rollout. We then supervise the predicted velocity with a corrective target determined by the rollout state and the desired clean edit $\mathbf{z}_0$:

$$\mathcal{L}^{(k)}_{\mathrm{rec}} = \left\| f_\theta(\mathbf{z}^{(k)}, \sigma_k, c) - \mathrm{sg}\left(\frac{\mathbf{z}^{(k)} - \mathbf{z}_0}{\sigma_k} \right) \right\|_2^2, \tag{4}$$

where sg denotes stop-gradient. This target reduces to the standard Flow Matching velocity $\boldsymbol{\epsilon} - \mathbf{z}_0$ when $\mathbf{z}^{(k)}$ lies on the target–noise interpolation path, but provides corrective supervision when the rollout deviates from it. This enables the model to learn to recover from its own trajectory deviations during denoising.

Table 1 Unified audio editing performance across models on the Complex Audio Editing Test Set. Our Base Editor denotes the pretrained model before STEER; SteerSound denotes our final model after both STEER stages. "–" in the *Train.* column denotes training-free methods built upon existing pretrained audio generation models, while "–" in task columns indicates that the model or method does not support the corresponding task.

	Params.		Generate		Add		Replace		Remove			Pitch			Volume		
Model	Total	Train.	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	LSD↓	KL↓	MAE↓	LSD↓	KL↓	MAE↓	LSD↓	KL↓
Audio-Omni	7.907B	3.047B	0.231	2.057	0.196	1.955	0.154	2.561	0.001	1.695	2.803	–	–	–	–	–	–
MMEdit	9.057B	0.616B	–	–	0.159	1.882	0.085	2.706	0.012	2.060	2.155	–	–	–	14.043	1.042	0.427
AUDIT	1.067B	0.860B	–	–	0.151	1.417	0.082	1.897	0.038	1.603	2.691	–	–	–	–	–	–
PPAE	1.318B	–	–	–	0.182	1.566	0.094	1.501	0.044	1.539	1.384	–	–	–	–	–	–
SDEdit	1.118B	–	–	–	0.111	2.347	0.129	2.222	0.129	1.778	1.969	0.923	0.942	1.963	5.047	0.895	1.992
ZETA	1.118B	–	–	–	0.135	2.060	0.140	1.939	0.135	1.688	1.690	0.570	0.803	1.441	4.199	0.722	1.432
SAM Audio(Small)	0.714B	0.492B	–	–	–	–	–	–	0.059	1.514	1.726	–	–	–	–	–	–
SAM Audio(Large)	3.154B	2.929B	–	–	–	–	–	–	0.056	1.465	1.629	–	–	–	–	–	–
UNISON	8.086B	0.732B	**0.247**	**1.845**	0.250	**1.218**	0.222	1.783	0.112	1.576	1.564	–	–	–	–	–	–
Our Base Editor	1.348B	0.673B	0.223	1.915	0.249	1.286	0.230	1.032	0.137	1.346	1.140	0.055	0.493	0.281	0.702	0.433	0.200
SteerSound	1.348B	0.673B	0.220	2.133	**0.251**	1.352	**0.235**	**1.028**	**0.144**	**1.164**	**0.788**	**0.047**	**0.491**	**0.280**	**0.634**	**0.433**	**0.200**

Table 2 Effectiveness of STEER across models on the Complex Audio Editing Test Set. TBG scores are computed relative to the corresponding base model. "–" indicates that the corresponding training stage is not applied or the editing task is not supported.

Model	**STEER-Emphasis**	**STEER-Recovery**	**Generate**		**Add**		**Replace**		**Remove**			**Pitch**			**Volume**			**Overall**	**Selective**
			CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	LSD↓	KL↓	MAE↓	LSD↓	KL↓	MAE↓	LSD↓	KL↓	TBG_{all} ↑	TBG_{sel} ↑
Our Base Editor	–	–	**0.22**	**1.92**	**0.25**	**1.29**	**0.23**	**1.03**	**0.14**	1.35	1.14	0.06	**0.49**	**0.28**	0.70	**0.43**	**0.20**	0.00	0.00
+Emph. only	✓	–	**0.22**	2.12	**0.25**	1.32	**0.23**	1.05	**0.14**	1.22	0.90	**0.05**	**0.49**	**0.28**	0.71	**0.43**	**0.20**	1.67	2.97
+Full STEER	✓	✓	**0.22**	2.13	**0.25**	1.35	**0.23**	**1.03**	**0.14**	**1.16**	**0.79**	**0.05**	**0.49**	**0.28**	**0.63**	**0.43**	**0.20**	**5.36**	**8.25**
UNISON	–	–	**0.25**	1.85	0.25	**1.22**	0.22	1.78	0.11	1.58	1.56	–	–	–	–	–	–	0.00	0.00
+Emph. only	✓	–	**0.25**	1.84	0.26	1.37	**0.24**	1.16	**0.15**	1.39	0.89	–	–	–	–	–	–	10.27	17.85
+Full STEER	✓	✓	0.25	**1.83**	**0.27**	1.32	0.23	**1.15**	**0.15**	**1.36**	**0.82**	–	–	–	–	–	–	**10.81**	**18.42**
SAM Audio	–	–	–	–	–	–	–	–	0.06	1.51	1.73	–	–	–	–	–	–	0.00	0.00
+Emph. only	✓	–	–	–	–	–	–	–	0.12	1.43	1.14	–	–	–	–	–	–	108.62	108.62
+Full STEER	✓	✓	–	–	–	–	–	–	**0.13**	**1.27**	**0.97**	–	–	–	–	–	–	**124.53**	**124.53**

We progressively increase the rollout depth k from 1, initializing each depth from the best checkpoint found so far. Generate and Add retain the standard Flow Matching objective throughout this stage. Training stops after P consecutive depths without improvement in the best validation score, and the best checkpoint is retained. Algorithm 1 summarizes the complete STEER procedure. We obtain SteerSound by applying both stages of STEER to our base editor.

5 Experiments

We first describe the experimental configuration and compare unified audio editing performance across models. We then evaluate STEER's effectiveness, conduct ablation studies on its two post-training stages, and examine its robustness to alternative definitions of the critical early-trajectory range. Next, we assess STEER's generalizability across model architectures for audio editing. Finally, we investigate whether early-trajectory emphasis also benefits related selective-control tasks.

5.1 Experimental Configuration

Base Editor. As shown in Figure 6, our base editor adopts a SeedEdit-inspired dual-branch DiT [29]. The reference and generated audio branches exchange information through multimodal self-attention with shared attention parameters, and both receive timestep and task-specific conditioning. The model combines a 673.35M-parameter trainable backbone with a T5-Base [25] text encoder, an audio VAE [26] trained from scratch, and a HiFi-GAN [13] vocoder. Further training details are provided in Appendix B.

Data, Benchmark, and Metrics. We use two complementary data settings. To better approximate real-world audio editing scenarios, we construct Complex Audio Editing Data with diverse multi-source compositions. However, source overlap in these scenes makes selective editing failures difficult to identify automatically. To

Table 3 Effectiveness of STEER across models on the Selective Audio Editing Benchmark. Succ. and SErr. denote success and selective error rates, respectively. "–" indicates that the corresponding training stage is not applied or the editing task is not supported.

Model	STEER-Emphasis	STEER-Recovery	Generate	Add	Replace		Remove		Pitch		Volume		Overall	
			Succ.↑	Succ.↑	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓
Our Base Editor	–	–	92.80	93.20	67.90	24.10	72.50	22.80	51.20	48.40	39.10	24.70	70.42	28.48
+Emph. only	✓	–	94.00	**94.60**	78.90	14.00	88.10	**7.90**	87.90	11.70	61.60	6.80	84.02	9.99
+Full STEER	✓	✓	**94.80**	93.50	**79.10**	**12.30**	**88.20**	**7.90**	**92.80**	**6.80**	**68.00**	**6.40**	**85.78**	**8.45**
UNISON	–	–	**96.40**	**90.10**	33.40	50.90	25.30	26.90	–	–	–	–	61.30	38.90
+Emph. only	✓	–	94.80	85.60	78.40	13.60	38.30	8.50	–	–	–	–	74.28	11.05
+Full STEER	✓	✓	94.90	85.40	**79.60**	**13.10**	**41.10**	**8.00**	–	–	–	–	**75.25**	**10.55**
SAM Audio	–	–	–	–	–	–	37.60	55.90	–	–	–	–	37.60	55.90
+Emph. only	✓	–	–	–	–	–	80.60	19.10	–	–	–	–	80.60	19.10
+Full STEER	✓	✓	–	–	–	–	**84.10**	**15.70**	–	–	–	–	**84.10**	**15.70**

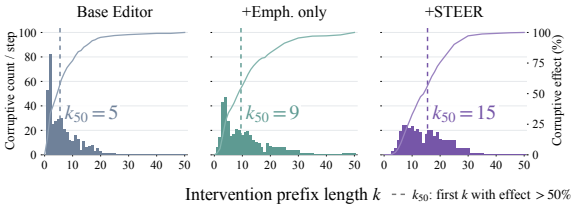


Figure 8 Recovery capability under early-trajectory corruption for the base editor, the STEER-Emphasis-only variant, and the full STEER variant.

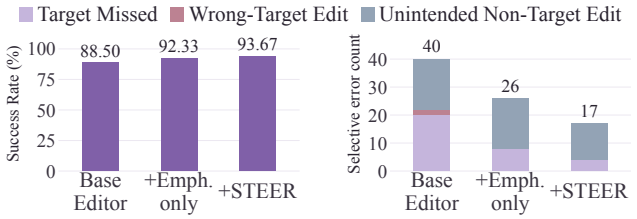


Figure 9 Human evaluation of our base editor, its Emphasis-only variant, and full STEER variant on the Complex Audio Editing Test Set, 100 random samples per task.

support systematic analysis and large-scale experiments, we introduce the Selective Audio Editing Benchmark. Its deliberately designed source composition allows target manipulation and non-target preservation to be evaluated separately, enabling automatic identification and measurement of selective editing failures. We also construct training and validation sets from the same distribution. For experiments, results reported on each test set are obtained from models post-trained on its corresponding training set, with checkpoints selected using the corresponding validation set. Figure 7 presents statistics and representative examples.

On the complex test set, we report task-aware CLAP Score, pitch and volume MAE, LSD [32], and KL-softmax [18]. To summarize overall unified editing performance, we introduce the Task-Balanced Gain (TBG_{all}) to measure a model's overall unified editing capability, and TBG_{sel} to specifically quantify its selective editing capability. On the selective audio editing benchmark, we develop an evaluation protocol that measures overall editing success while identifying and categorizing different types of source-selection errors. Further details on the two datasets and their corresponding metrics are provided in Appendices C and D respectively.

Implementation Details. Across models, we use the critical early-trajectory ranges defined in Section 4.2.1 for STEER-Emphasis and set the patience hyperparameter to $P = 2$ for STEER-Recovery. Unless otherwise specified, our base editor and its post-trained variants use classifier-free guidance (CFG) at inference, with guidance scales of 7.5 for source addition and 3.5 for the other tasks, while other models retain their official default configurations. Experiment-specific hyperparameters, training procedures, and learning dynamics are provided in the corresponding appendices.

5.2 Unified Audio Editing Performance

We compare SteerSound with eight existing audio editors on the Complex Audio Editing Test Set, as shown in Table 1. SteerSound supports all six tasks, whereas existing editors cover only subsets, and achieves the best primary metric on five tasks, including all four selective editing tasks. Notably, it outperforms substantially larger models on removal and replacement while achieving the lowest pitch and volume errors. Compared with our already competitive base editor, STEER further improves removal and replacement CLAP scores

Table 4 STEER-Emphasis versus standard Flow Matching post-training under matched updates on both sets.

Backbone	Strategy	Complex		Selective	
		TBG_{all} ↑	TBG_{sel} ↑	Succ.↑	SErr.↓
Our Base Editor	Standard FM	0.92	0.66	81.05	13.58
	STEER-Emphasis	**1.67**	**2.97**	**84.02**	**9.99**
UNISON	Standard FM	6.97	12.89	68.18	18.95
	STEER-Emphasis	**10.27**	**17.85**	**74.28**	**11.05**
SAM Audio	Standard FM	65.69	65.69	50.50	49.40
	STEER-Emphasis	**108.62**	**108.62**	**80.60**	**19.10**

Table 5 STEER-Recovery versus continued STEER-Emphasis under matched updates on both sets.

Backbone	Strategy	Complex		Selective	
		TBG_{all} ↑	TBG_{sel} ↑	Succ.↑	SErr.↓
Our Base Editor	Emphasis (continue)	3.61	5.50	84.51	8.67
	STEER-Recovery	**5.36**	**8.25**	**85.78**	**8.45**
UNISON	Emphasis (continue)	10.27	17.85	74.28	11.05
	STEER-Recovery	**10.81**	**18.42**	**75.25**	**10.55**
SAM Audio	Emphasis (continue)	105.08	105.08	82.10	17.50
	STEER-Recovery	**124.53**	**124.53**	**84.10**	**15.70**

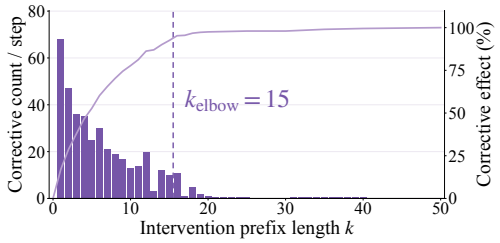


Figure 10 Corrective effect and per-step counts, with the elbow at $k = 15$ defining the shared early range.

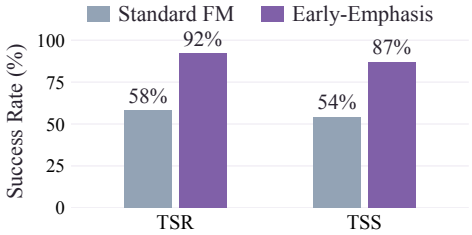


Figure 11 Human evaluation of standard Flow Matching and early-trajectory emphasis on TSR and TSS.

from 0.137 and 0.230 to 0.144 and 0.235, while reducing pitch and volume MAEs from 0.055 and 0.702 to 0.047 and 0.634. These results demonstrate SteerSound's comprehensive task coverage and strong selective editing performance, further enhanced by STEER.

5.3 Effectiveness of STEER across Models and Datasets

We evaluate STEER on our base editor, UNISON, and SAM Audio using the Complex Audio Editing Test Set and the Selective Audio Editing Benchmark. As shown in Tables 2 and 3, STEER improves aggregate editing performance across all three models in both settings. On the selective benchmark, it increases overall success rates while reducing selective error rates by over 70% relative to each corresponding base model. STEER-Emphasis consistently improves performance, and adding STEER-Recovery yields further gains. These results demonstrate the effectiveness of STEER across different editing backbones under both controlled source compositions and complex audio scenes. Further experimental details and results are provided in Appendix E.

5.4 Analysis of Source-Selection Reliability

Recovery from Early-Trajectory Errors. Following the corruptive prefix intervention protocol in Section 3.2, we evaluate recovery capability across three model variants on the Selective Audio Editing Benchmark. As shown in Figure 8, the prefix length required to induce a 50% failure rate increases from 5 steps for our base editor to 9 after STEER-Emphasis and 15 after full STEER. These results demonstrate that STEER-Emphasis improves the model's ability to recover from early source-selection errors, while STEER-Recovery further strengthens this capability. Further experimental results are provided in Appendix F.1.

Human Evaluation. To assess source-selection reliability in complex audio scenes, we conduct human evaluation on 100 randomly sampled examples per task from the Complex Audio Editing Test Set. Annotators assess whether the intended source is correctly edited while non-target content is preserved. As shown in Figure 9, overall success increases from 88.50% to 92.33% after STEER-Emphasis and 93.67% after full STEER, while selective error counts decrease from 40 to 26 and 17, respectively. These results further confirm improved source-selection reliability in complex audio scenes. Detailed results are provided in Appendix F.2.

Table 6 Comparison of task-group-specific and elbow-based shared early trajectory ranges for STEER-Emphasis on both evaluation sets.

Backbone	Strategy	Complex		Selective	
		TBG_{all} ↑	TBG_{sel} ↑	Succ.↑	SErr.↓
Our Base Editor	$Emph._{elb.}$	1.36	2.82	83.98	**9.68**
	Emphasis	**1.67**	**2.97**	**84.02**	9.99

Table 7 Evaluation of standard Flow Matching and early-trajectory emphasis on TSR and TSS using our base editor and SAM Audio.

Method	Our Base Editor	SAM Audio
	$CLAP\text{-}AA_{TSR}$ ↑	$CLAP\text{-}AA_{TSS}$ ↑
Standard FM	0.74	0.00
Early-Emphasis	**0.86**	**0.21**

5.5 Ablation Studies

Effectiveness of the Two STEER Stages. We compare STEER-Emphasis with standard Flow Matching post-training and STEER-Recovery with continued STEER-Emphasis, using matched numbers of optimizer updates within each comparison. Within each comparison, both runs start from the same checkpoint, with comparable training settings. Tables 4 and 5 show that both stages outperform their respective controls across all three models and both evaluation settings. These results demonstrate the effectiveness of both STEER stages beyond simply increasing the number of training updates. Further experimental details and results are provided in Appendix G.1 and G.2.

Alternative Early-Trajectory Range Selection. To examine whether the benefits of STEER-Emphasis stem from emphasizing critical early states rather than task-specific range selection, we evaluate an alternative that uses a single shared range across all selective tasks. Specifically, instead of using task-group-specific ranges determined by the 80% cumulative-effect threshold, we average the cumulative corrective effects across selective tasks and identify the point beyond which longer corrective prefixes provide diminishing gains (the elbow). As shown in Figure 10, this gives a shared range covering the first 15 denoising steps, which we use for uniform noise-level sampling across all selective tasks. Table 6 shows that this alternative also improves editing performance on both datasets, increasing the overall success rate from 70.42% to 83.98% and reducing selective errors from 28.48% to 9.68% on the selective benchmark. Although our task-group-specific design achieves slightly higher task-balanced gains on the complex test set, both strategies provide substantial improvements. These results suggest that the benefits of STEER-Emphasis stem from emphasizing critical early states rather than task-specific range selection. Further details are provided in Appendix G.3.

5.6 Extension to Fine-Grained Selective Control

We further examine whether early-trajectory emphasis extends beyond category-level audio editing to related selective-control tasks. To this end, we study transcript-conditioned target speaker removal (TSR) and target speaker separation (TSS), adapting our base editor for TSR and SAM Audio for TSS. These tasks require distinguishing competing speakers by speech content rather than sound category. We compare standard Flow Matching training with early-trajectory emphasis from the start of task adaptation. The quantitative results in Table 7 and human evaluation on 100 randomly sampled test examples per task in Figure 11 show substantial gains on both tasks, with early-trajectory emphasis enabling effective target selection where standard training struggles. These findings support the effectiveness of early-trajectory emphasis for fine-grained source selection and its broader applicability to related selective-control tasks. Further details are provided in Appendix H.

6 Discussion

Our analysis shows that source selection in audio editing is largely shaped during early denoising, and early selection errors often persist through subsequent steps. STEER addresses this behavior through early-trajectory emphasis and recovery training on self-generated rollout states. Extensive experiments across different model architectures demonstrate consistent improvements in selective editing reliability and overall editing performance. Furthermore, the effectiveness of early-trajectory emphasis on transcript-conditioned speaker removal and separation suggests its applicability to broader fine-grained selective-control tasks.

These findings suggest that training for selective control should account for both when consequential decisions are made and how models respond to trajectory deviations. Preventing early errors and recovering from errors

that remain are complementary needs. Beyond audio, our findings suggest a general training principle for iterative generative models: improving early decisions and learning to recover from trajectory deviations may enhance the reliability of final outputs. This perspective offers a promising direction for image and video generation and editing, where early decisions about scene structure and target selection may influence results.

References

[1] Yogesh Balaji, Seungjun Nah, Xun Huang, Arash Vahdat, Jiaming Song, Qinsheng Zhang, Karsten Kreis, Miika Aittala, Timo Aila, Samuli Laine, et al. ediff-i: Text-to-image diffusion models with an ensemble of expert denoisers. arXiv preprint arXiv:2211.01324, 2022.

[2] Dong Chen, Fangyun Wei, Ziyu Wan, Dongdong Chen, Jiawei Zhang, Jinjing Zhao, Sirui Zhang, Yang Yue, Zhiyang Liang, Baining Guo, et al. Lens: Rethinking training efficiency for foundational text-to-image models. arXiv preprint arXiv:2605.21573, 2026.

[3] Honglie Chen, Weidi Xie, Andrea Vedaldi, and Andrew Zisserman. Vggsound: A large-scale audio-visual dataset. In ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 721–725. IEEE, 2020.

[4] Konstantinos Drossos, Samuel Lipping, and Tuomas Virtanen. Clotho: An audio captioning dataset. In ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 736–740. IEEE, 2020.

[5] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. Clap learning audio concepts from natural language supervision. In ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 1–5. IEEE, 2023.

[6] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In Forty-first international conference on machine learning, 2024.

[7] Eduardo Fonseca, Xavier Favory, Jordi Pons, Frederic Font, and Xavier Serra. Fsd50k: an open dataset of human-labeled sound events. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 30:829–852, 2021.

[8] Tiankai Hang, Shuyang Gu, Chen Li, Jianmin Bao, Dong Chen, Han Hu, Xin Geng, and Baining Guo. Efficient diffusion training via min-snr weighting strategy. In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pages 7407–7417. IEEE, 2023.

[9] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.

[10] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. Advances in Neural Information Processing Systems, 38:167283–167308, 2026.

[11] Yuhang Jia, Yang Chen, Jinghua Zhao, Shiwan Zhao, Wenjia Zeng, Yong Chen, and Yong Qin. Audioeditor: A training-free diffusion-based audio editing framework. In ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 1–5. IEEE, 2025.

[12] Chris Dongjoo Kim, Byeongchang Kim, Hyunmin Lee, and Gunhee Kim. Audiocaps: Generating captions for audios in the wild. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 119–132, 2019.

[13] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. Advances in neural information processing systems, 33:17022–17033, 2020.

[14] Khaled Koutini, Jan Schlüter, Hamid Eghbal-Zadeh, and Gerhard Widmer. Efficient training of audio transformers with patchout. arXiv preprint arXiv:2110.05069, 2021.

[15] Kai Li, Jintao Cheng, Chang Zeng, Zijun Yan, Helin Wang, Zixiong Su, Bo Zheng, and Xiaolin Hu. A semantically consistent dataset for data-efficient query-based universal sound separation. arXiv preprint arXiv:2601.22599, 2026.

[16] Zhaoqing Li, Haoning Xu, Jingran Su, Yaofang Liu, Zhefan Rao, Huimeng Wang, Jiajun Deng, Tianzi Wang, Zengrui Jin, Rui Liu, et al. Unison: A unified sound generation and editing framework via deep llm fusion. arXiv preprint arXiv:2605.31530, 2026.

[17] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. arXiv preprint arXiv:2210.02747, 2022.

[18] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D Plumbley. Audioldm: Text-to-audio generation with latent diffusion models. arXiv preprint arXiv:2301.12503, 2023.

[19] Hila Manor and Tomer Michaeli. Zero-shot unsupervised and text-based audio editing using ddpm inversion. arXiv preprint arXiv:2402.10009, 2024.

[20] Irene Martin Morato and Annamaria Mesaros. Macs-multi-annotator captioned soundscapes. MACS-Multi-Annotator Captioned Soundscapes, 2021.

[21] Xichen Pan, Satya Narayan Shukla, Aashu Singh, Zhuokai Zhao, Shlok Kumar Mishra, Jialiang Wang, Zhiyang Xu, Jiuhai Chen, Kunpeng Li, Felix Juefei-Xu, et al. Transfer between modalities with metaqueries. arXiv preprint arXiv:2504.06256, 2025.

[22] Ziqi Pang, Xin Xu, and Yu-Xiong Wang. Aligning generative denoising with discriminative objectives unleashes diffusion for visual perception. In International Conference on Learning Representations, volume 2025, pages 101991–102013, 2025.

[23] William Peebles and Saining Xie. Scalable diffusion models with transformers. In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pages 4172–4182. IEEE, 2023.

[24] You Qin, Linqing Wang, Hao Fei, Roger Zimmermann, Liefeng Bo, Qinglin Lu, and Chunyu Wang. Soar: Self-correction for optimal alignment and refinement in diffusion models. arXiv preprint arXiv:2604.12617, 2026.

[25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140):1–67, 2020.

[26] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In 2022 IEEE/CVF conference on computer vision and pattern recognition (CVPR), pages 10674–10685. ieee, 2022.

[27] Kentaro Seki, Nobutaka Ito, Kazuki Yamauchi, Yuki Okamoto, Kouei Yamaoka, Yuki Saito, Shinnosuke Takamichi, and Hiroshi Saruwatari. Language-queried target speech extraction using para-linguistic and non-linguistic prompts. Acoustical Science and Technology, pages e25–27, 2025.

[28] Bowen Shi, Andros Tjandra, John Hoffman, Helin Wang, Yi-Chiao Wu, Luya Gao, Julius Richter, Matt Le, Apoorv Vyas, Sanyuan Chen, et al. Sam audio: Segment anything in audio. arXiv preprint arXiv:2512.18099, 2025.

[29] Yichun Shi, Peng Wang, and Weilin Huang. Seededit: Align image re-generation to image editing. arXiv preprint arXiv:2411.06686, 2024.

[30] Ye Tao, Wen Wu, Chao Zhang, Mengyue Wu, Shuai Wang, and Xuenan Xu. Mmedit: A unified framework for multi-type audio editing via audio language model. arXiv preprint arXiv:2512.20339, 2025.

[31] Zeyue Tian, Binxin Yang, Zhaoyang Liu, Jiexuan Zhang, Ruibin Yuan, Hubery Yin, Qifeng Chen, Chen Li, Jing Lyu, Wei Xue, et al. Audio-omni: Extending multi-modal understanding to versatile audio generation and editing. arXiv preprint arXiv:2604.10708, 2026.

[32] Yuancheng Wang, Zeqian Ju, Xu Tan, Lei He, Zhizheng Wu, Jiang Bian, et al. Audit: Audio editing by following instructions with latent diffusion models. Advances in Neural Information Processing Systems, 36:71340–71357, 2023.

[33] Manjie Xu, Chenxing Li, Dan Su, Wei Liang, Dong Yu, et al. Prompt-guided precise audio editing with diffusion models. arXiv preprint arXiv:2406.04350, 2024.

[34] Chen Yang, Chufan Yu, Hanfu Chen, Jie Zhu, Jingqi Chen, Ke Chen, Wenxuan Wang, Yang Wang, Yaozhou Jiang, Yi Jiang, et al. Moss-audio technical report. arXiv preprint arXiv:2606.01802, 2026.

Appendix

A Early-trajectory experiments on our base editor

This appendix presents the early-trajectory analyses on our base editor. While Section 3 establishes the early formation and causal influence of source selection on UNISON and SAM Audio, here we extend the same analyses to the four selective editing tasks supported by our base editor: removal, replacement, pitch modification, and volume modification. We first examine when erroneous source-selection tendencies become observable, and then use bidirectional prefix interventions to identify the corresponding source-selection-critical early ranges.

A.1 Early-Trajectory Observation

Figure 12 shows representative cases. Across all four tasks, erroneous source-selection tendencies become perceptually distinguishable during the early trajectory. Removal reveals the selected source within only a few steps, whereas replacement and source-attribute modifications require a broader, but still early, portion of the trajectory before their editing tendencies become clearly distinguishable. These observations are consistent with the task-dependent behavior discussed in Section 3.1.

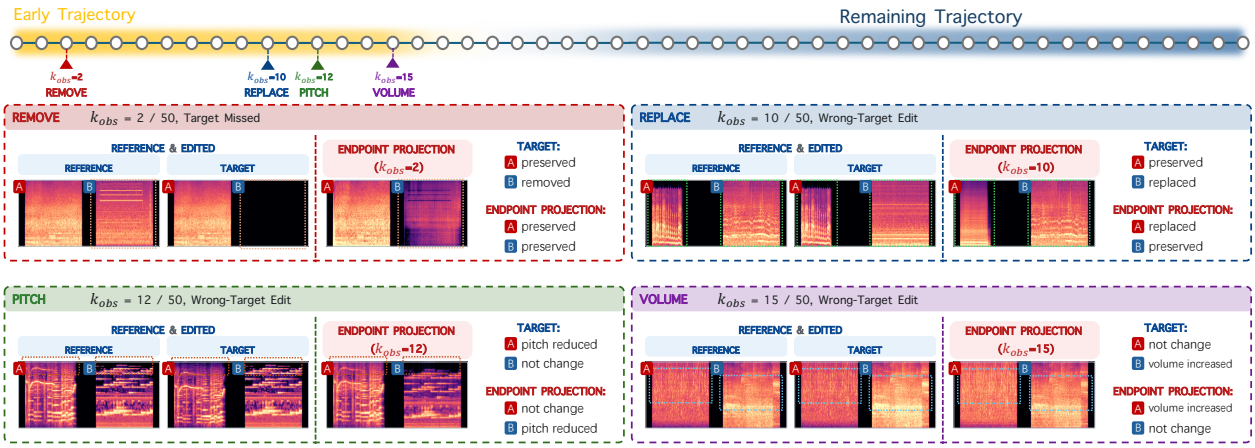


Figure 12 Early emergence of source-selection tendencies on our base editor. Intermediate denoising states are extrapolated to estimated zero-noise outputs. Representative failed edits from the four selective editing tasks are shown, with k_{obs} denoting the earliest step at which the erroneous source-selection tendency becomes perceptually distinguishable.

A.2 Bidirectional Early-Trajectory Intervention

Following the protocol in Section 3.2, we perform the same bidirectional prefix interventions on our base editor using 100 validation examples per selective editing task. As shown in Figure 13, the intervention effects are likewise concentrated in the early trajectory, with removal exhibiting a substantially shorter sensitive range than replacement, pitch modification, and volume modification. Applying the cumulative-corrective-effect criterion defined in Section 4.2.1 with $\rho = 0.8$ yields $k^* = 2$ of 50 steps for selective suppression and $k^* = 15$ for selective transformation. These ranges are used by STEER-Emphasis.

B Base editor architecture and training details

This appendix complements Section 5.1 with additional implementation details of our base editor. As introduced in the main paper, our base editor serves as the common backbone for our trajectory analysis and subsequent STEER post-training. Table 8 summarizes the parameter composition of the complete system and its trainable editing backbone.

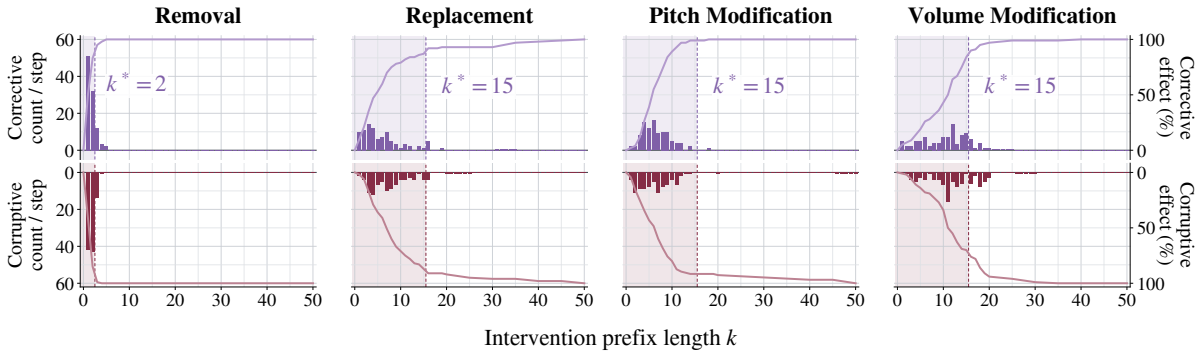


Figure 13 Bidirectional early-trajectory interventions on our base editor. Corrective (top) and corruptive (bottom) interventions are evaluated on 100 validation examples per selective editing task. Bars denote per-step intervention counts and curves denote cumulative effects. Dashed lines indicate the resulting critical ranges used by STEER-Emphasis: $k^* = 2$ for selective suppression and $k^* = 15$ for selective transformation.

Here, we focus on the implementation details omitted from the main text, including the conditioning mechanism, base-model optimization, and task-dependent inference configuration. We first describe how task-specific scalar controls are incorporated into the denoising backbone. Then we provide the optimization and learning-rate configuration used to obtain our base editor, followed by the construction of task-dependent CFG reference prompts at inference time.

Table 8 Parameter counts by module for our base editor.

Module	Params. (M)
DiT backbone	673.35
T5-Base	109.63
Audio VAE	509.34
HiFi-GAN	55.26
Total	1,347.58
Trainable	673.35

B.1 Task and Timestep Conditioning

In addition to the text instruction, the backbone receives four scalar conditions: the noise level σ, task ID τ, volume change Δ_v, and pitch change Δ_p. To combine denoising and editing information, each input is normalized to $[0, 1]$ and passed through sinusoidal encoding followed by an independent MLP. The four equal-dimensional embeddings are then summed:

$$\mathbf{c}_{\text{global}} = g_\sigma(\phi(\bar{\sigma})) + g_\tau(\phi(\bar{\tau})) + g_v\big(\phi(\overline{\Delta_v})\big) + g_p\big(\phi(\overline{\Delta_p})\big)\,, \tag{5}$$

where bars denote normalization, ϕ is sinusoidal encoding, and each g is an independent MLP. The resulting vector modulates intermediate features in both audio branches, following DiT and SD3 [6, 23], while T5 provides text-token conditioning separately. Notably, in our model, we set the pitch-change range Δ_p to $[-4, +4]$ semitones with a step size of 0.5 semitones, and the volume-change range Δ_v to $[-5, +5]$ dB. The signs specify the direction of the requested change.

B.2 Training Configuration

To train this conditioned backbone, we use the Flow Matching objective in Equation 1, starting from random initialization. During pre-training, we adopt the standard logit-normal noise-level sampling strategy following SD3 [6]. We use AdamW with $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$, and zero weight decay. The learning rate reaches 10^{-4} after 1,000 warmup steps and subsequently follows a cosine decay schedule. Training uses BF16 precision and DeepSpeed ZeRO-2 for 15 epochs on our Complex Audio Editing Training Set and takes approximately 16 hours on 16 Ascend 910B NPUs. Figure 14 reports the training loss and learning-rate schedule.

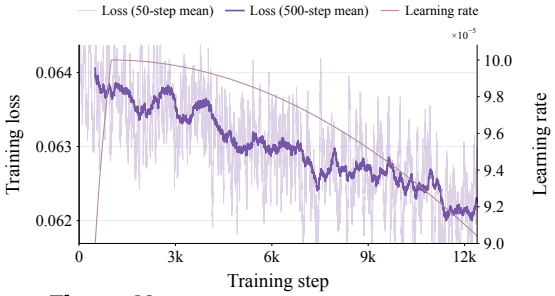


Figure 14 Training loss and learning rate.

B.3 Task-Dependent Inference Prompts

For inference, we use the CFG configuration [9] with task-dependent reference prompts (Table 9). Generate, Add, Remove, and Replace use an empty string. In contrast, Volume and Pitch retain the direction and magnitude but omit the source description. Attribute magnitudes are absolute values with one decimal place, and Increase or Reduce specifies the direction. Units are implicit: dB for Volume and semitones for Pitch.

Table 9 Example conditional and CFG reference prompts.

Task	Conditional prompt	CFG reference prompt
Generate	Generate the sound of rain.	*Empty string*
Add	Add the sound of rain.	*Empty string*
Remove	Remove the sound of rain.	*Empty string*
Replace	Replace the sound of rain by the sound of wind.	*Empty string*
Volume ↑	Increase the volume of the sound of rain by 3.0.	Increase the volume by 3.0.
Volume ↓	Reduce the volume of the sound of rain by 3.0.	Reduce the volume by 3.0.
Pitch ↑	Increase the pitch of the sound of rain by 2.0.	Increase the pitch by 2.0.
Pitch ↓	Reduce the pitch of the sound of rain by 2.0.	Reduce the pitch by 2.0.

C Complex Audio Editing Data: Construction and Evaluation

C.1 Audio Data Sources

Task-Adaptive Caption Granularity. As described in Section 4.1, we consider six editing operations: generation, addition, removal, replacement, pitch modification, and volume modification. Through extensive experiments, we observe that different editing operations benefit from different granularities of textual supervision. Specifically, audio generation and source addition perform better with fine-grained audio descriptions, whereas source removal, pitch modification, and volume modification benefit more from class-level descriptions. This difference can be understood from their training and inference settings: fine-grained descriptions provide richer semantic constraints for generation-oriented operations, while concise class labels establish a clearer correspondence between semantic categories and audio sources, which better matches selective editing scenarios where the target source is typically specified by a category term. Similar benefits of information-rich captions have also been observed in image generation [2].

Based on this observation, we use two separate source pools constructed from different types of annotated audio data: the Fine-Grained Description Pool $\mathcal{P}$ and the Class-Label Pool $\mathcal{L}$. The former consists of audio paired with fine-grained natural-language descriptions, while the latter consists of single-source audio paired with class-level labels, where each clip contains only the labeled sound source. Samples from $\mathcal{P}$ are used for audio generation, source addition, and the generated content in source replacement, whereas samples from $\mathcal{L}$ are used for source removal, pitch modification, volume modification, and the source to be replaced in source replacement.

The Fine-Grained Description Pool $\mathcal{P}$ is collected from VGGSound [3], AudioCaps [12], FSD50K [7], Clotho V2 [4], and MACS [20]. It contains 380,165 audio chunks with a total duration of 953.51 hours, with each sample paired with a fine-grained description of its acoustic content. The data composition across source datasets is summarized in Table 10. The Class-Label Pool $\mathcal{L}$ is constructed from HIVE [15] by retaining audio-label pairs with CLAP similarity above 0.1. It contains 635,019 audio chunks with a total duration of 1,688.44 hours and covers 279 sound categories.

Table 10 Data sources for $\mathcal{P}$.

Dataset	# Chunks
VGGSound	192,794
AudioCaps	101,369
FSD50K	51,182
Clotho V2	17,545
MACS	17,275
Total	**380,165**

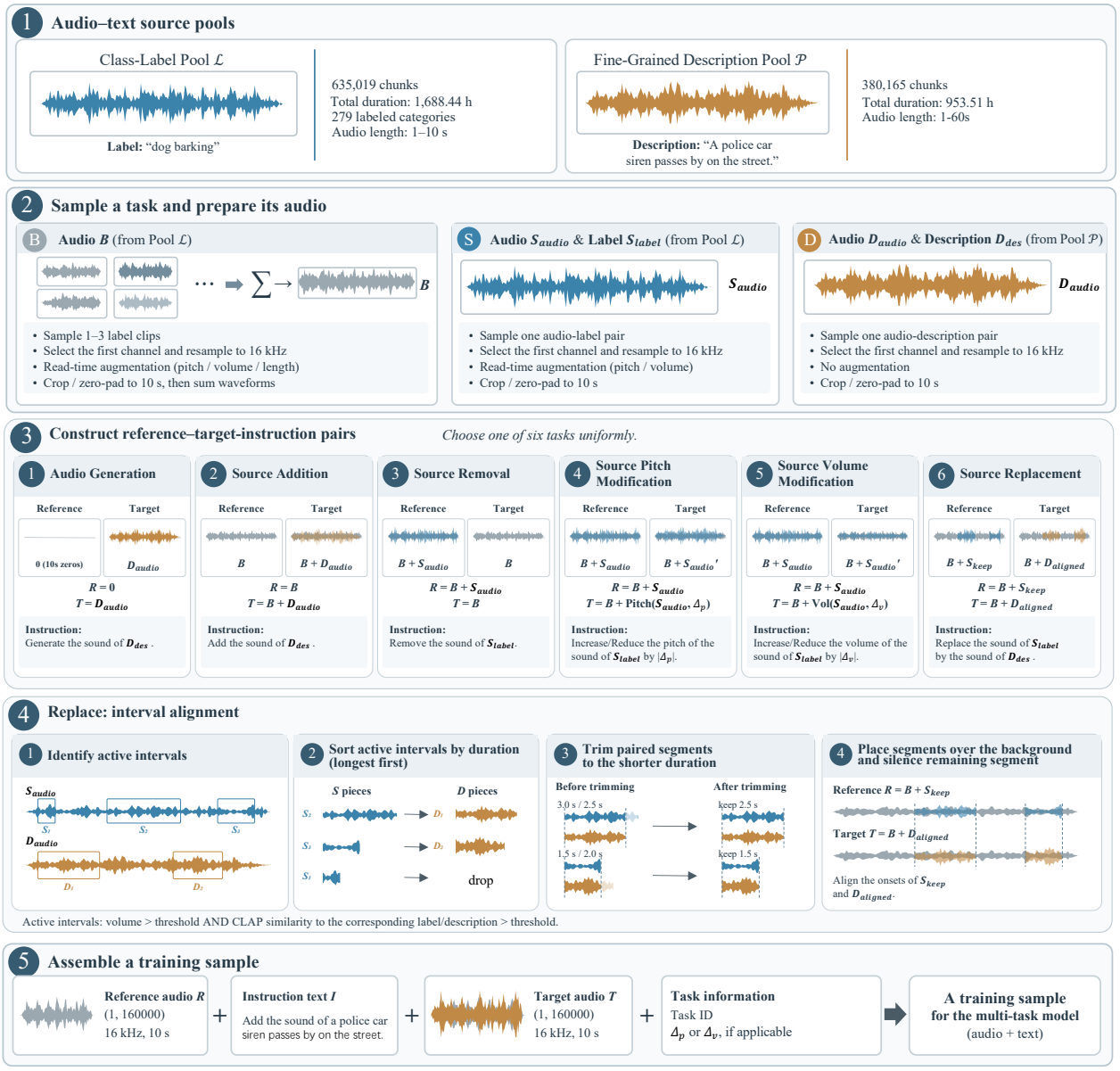


Figure 15 Overview of the online Complex Audio Editing Data constructor. Audio sources from the Class-Label Pool $\mathcal{L}$ and Fine-Grained Description Pool $\mathcal{P}$ are dynamically composed into reference–target–instruction pairs for six audio editing operations. $|\Delta_p|$ is capped at 4 semitones, and $|\Delta_v|$ is capped at 5 dB in the process.

C.2 Online Data Constructor

We develop the online Complex Audio Editing Data constructor to generate training data for the overall model. For each editing task, audio sources are sampled from different source pools following the principles described in Appendix B.1, and are then composed into the reference audio, target audio, and editing instruction. The detailed training data construction process is illustrated in Fig. 15.

Since training samples are constructed online, the training set is not restricted to a fixed collection of pre-generated editing pairs. We set its nominal length to approximately 600,000 samples. We additionally construct fixed validation and test sets, each containing 1,000 samples per operation and 6,000 samples in total. All audio sources appearing in the validation and test sets are excluded from training. Notably, during both training and evaluation, all audio samples are either cropped or padded to a fixed duration of 10 seconds.

C.3 Evaluation Metrics

We evaluate performance on Complex Audio Editing Data using task-aware semantic and attribute metrics, together with complementary audio-similarity metrics. For audio generation, source addition, source removal, and source replacement, we use a task-aware CLAP score as the primary metric. For source pitch and volume modification, the primary metrics are Pitch MAE and Volume MAE, respectively. We additionally report Log-Spectral Distance (LSD) and KL-softmax as complementary metrics.

Task-Aware CLAP. Task-aware CLAP is used to evaluate audio generation, source addition, source removal, and source replacement. These operations can naturally be characterized from three semantic aspects: whether the sources that should be preserved remain present, whether the sources that should be added are introduced, and whether the sources that should be removed are suppressed. We therefore design the task-aware CLAP score around these three aspects. Specifically, for each sample i, let c_i^P, c_i^A, and c_i^R denote the average CLAP similarities between the generated audio and the labels corresponding to sources that should be preserved, added, and removed, respectively. The task-aware CLAP score is defined as

$$s_i^{\text{CLAP}} = \begin{cases} c_i^A, & \text{Generate}, \\ c_i^P + c_i^A, & \text{Add}, \\ c_i^P - c_i^R, & \text{Remove}, \\ c_i^P + c_i^A - c_i^R, & \text{Replace}. \end{cases} \tag{6}$$

This formulation combines the semantic aspects involved in each operation to measure its overall task fulfillment. Higher values indicate better fulfillment of the task-specific semantic requirements, and the average score over all valid samples reflects the model's overall performance on the corresponding task.

Pitch and Volume MAE. For source pitch and volume modification, directly measuring the edited source would require first separating it from the generated mixture, which can introduce substantial measurement errors. We therefore evaluate these two operations indirectly by measuring the residual pitch and volume differences between the generated audio and the corresponding target audio without source separation. For each sample i, let $\widehat{\Delta}_i^{\text{pitch}}$ and $\widehat{\Delta}_i^{\text{volume}}$ denote the residual pitch and volume offsets, respectively. The pitch offset is estimated using bidirectional multi-resolution CQT alignment, while the volume offset is computed from the RMS-level difference between the generated and target audio. We define

$$\text{MAE}_q = \frac{1}{|\mathcal{V}_q|} \sum_{i \in \mathcal{V}_q} \left| \widehat{\Delta}_i^q \right|, \qquad q \in \{\text{pitch}, \text{volume}\}, \tag{7}$$

where $\mathcal{V}_q$ denotes the set of valid samples for metric q. Pitch MAE is measured in semitones and Volume MAE in dB, with lower values indicating better editing performance. The task-level result is obtained by averaging over all valid samples. Unlike volume, pitch estimation depends on the measurement method, and the pitch of some audio samples cannot be measured. We therefore select the pitch estimator that provides the most accurate measurements through extensive validation of multiple estimation methods, and exclude pitch-unmeasurable samples from $\mathcal{V}_{\text{pitch}}$. This measurability issue applies only to Pitch MAE.

LSD and KL-softmax. In addition to the task-related metrics above, we further evaluate the similarity between the generated and target audio from spectral and semantic perspectives. For source removal, pitch modification, and volume modification, the desired output is defined by a specific transformation of the input audio and is therefore expected to closely match the constructed target at the signal level. We thus report Log-Spectral Distance (LSD) for these three operations to measure their log-magnitude spectral discrepancy. For all six operations, we additionally report KL-softmax [18], which measures the KL divergence between the PaSST [14] audio-event distributions of the target and generated audio and reflects their similarity in semantic audio-event content. Lower values indicate greater similarity for both metrics.

Overall Task-Balanced Gain. Since multiple editing tasks share a single model, improvements on one task may be accompanied by degradation on others. Moreover, their evaluation metrics differ in scale and optimization

direction, making direct averaging inappropriate. We therefore introduce the overall Task-Balanced Gain, denoted as $\mathrm{TBG}_{\mathrm{all}}$. Let $\mathcal{T}_{\mathrm{all}}$ denote the set of all tasks supported by each model. Given a baseline model and one primary metric for each task, let M_t and M_t^{base} denote the corresponding metrics of the evaluated and baseline models, respectively. Here, the baseline models refer to the corresponding pretrained models for each architecture. We define

$$r_t = \begin{cases} \dfrac{M_t}{M_t^{\mathrm{base}}}, & t \in \{\mathrm{Generate}, \mathrm{Add}, \mathrm{Remove}, \mathrm{Replace}\}, \\ \dfrac{M_t^{\mathrm{base}}}{M_t}, & t \in \{\mathrm{Pitch}, \mathrm{Volume}\}, \end{cases} \tag{8}$$

such that $r_t > 1$ indicates improvement over the baseline. The overall score is computed from the equal-weight geometric mean across all supported tasks:

$$\mathrm{TBG}_{\mathrm{all}} = 100 \left[\left(\prod_{t \in \mathcal{T}_{\mathrm{all}}} r_t \right)^{1/|\mathcal{T}_{\mathrm{all}}|} - 1 \right]. \tag{9}$$

The geometric mean prevents gains on a subset of tasks from overly masking degradation on others. For our base editor, $\mathcal{T}_{\mathrm{all}}$ includes all six tasks, while for UNISON and SAM Audio, it includes their four and one supported tasks, respectively. We use Task-Aware CLAP as the primary metric for audio generation, source addition, source removal, and source replacement, Pitch MAE for pitch modification, and Volume MAE for volume modification.

Selective Task-Balanced Gain. To more directly assess the model's relative selective editing capability, we report a separate gain score over its supported selective editing tasks. Let $\mathcal{T}_{\mathrm{sel}} = \mathcal{T}_{\mathrm{all}} \cap \{\mathrm{Remove}, \mathrm{Replace}, \mathrm{Pitch}, \mathrm{Volume}\}$. Using the same baseline and task-level ratios r_t defined above, we compute

$$\mathrm{TBG}_{\mathrm{sel}} = 100 \left[\left(\prod_{t \in \mathcal{T}_{\mathrm{sel}}} r_t \right)^{1/|\mathcal{T}_{\mathrm{sel}}|} - 1 \right]. \tag{10}$$

Like $\mathrm{TBG}_{\mathrm{all}}$, this score is zero for the corresponding pretrained model, while positive values indicate aggregate improvement over the baseline.

D Selective Audio Editing Benchmark: Construction and Evaluation

D.1 Benchmark Construction

As discussed in Section 5.1, source overlap in natural acoustic scenes makes it difficult to automatically determine whether the intended source has been correctly edited while non-target content is preserved. We therefore construct the *Selective Audio Editing Benchmark* with controlled source composition and develop a corresponding evaluation protocol for identifying editing failures and selective failure types.

The benchmark is designed to enable large-scale automatic evaluation of both editing success and selective editing failure types under a controlled setting. For audio generation, we follow the same construction strategy as the Complex Audio Editing Data. For source addition, we restrict the reference audio to a single source. For selective editing operations, the reference audio always contains exactly two temporally non-overlapping sources. One serves as the target source to be edited, while the other serves as the non-target source to be preserved. This simplified and controlled source composition facilitates the evaluation of audio generation and source addition while allowing the editing behaviors of the target and non-target sources in selective editing operations to be measured independently.

We reuse the Class-Label Pool $\mathcal{L}$ and Fine-Grained Description Pool $\mathcal{P}$ introduced in Appendix B.1. We first construct task-specific source pools with different audio durations and then sample the required sources according to each editing task to form the reference audio R, target audio T, and editing instruction I. The overall construction procedure is illustrated in Fig. 16.

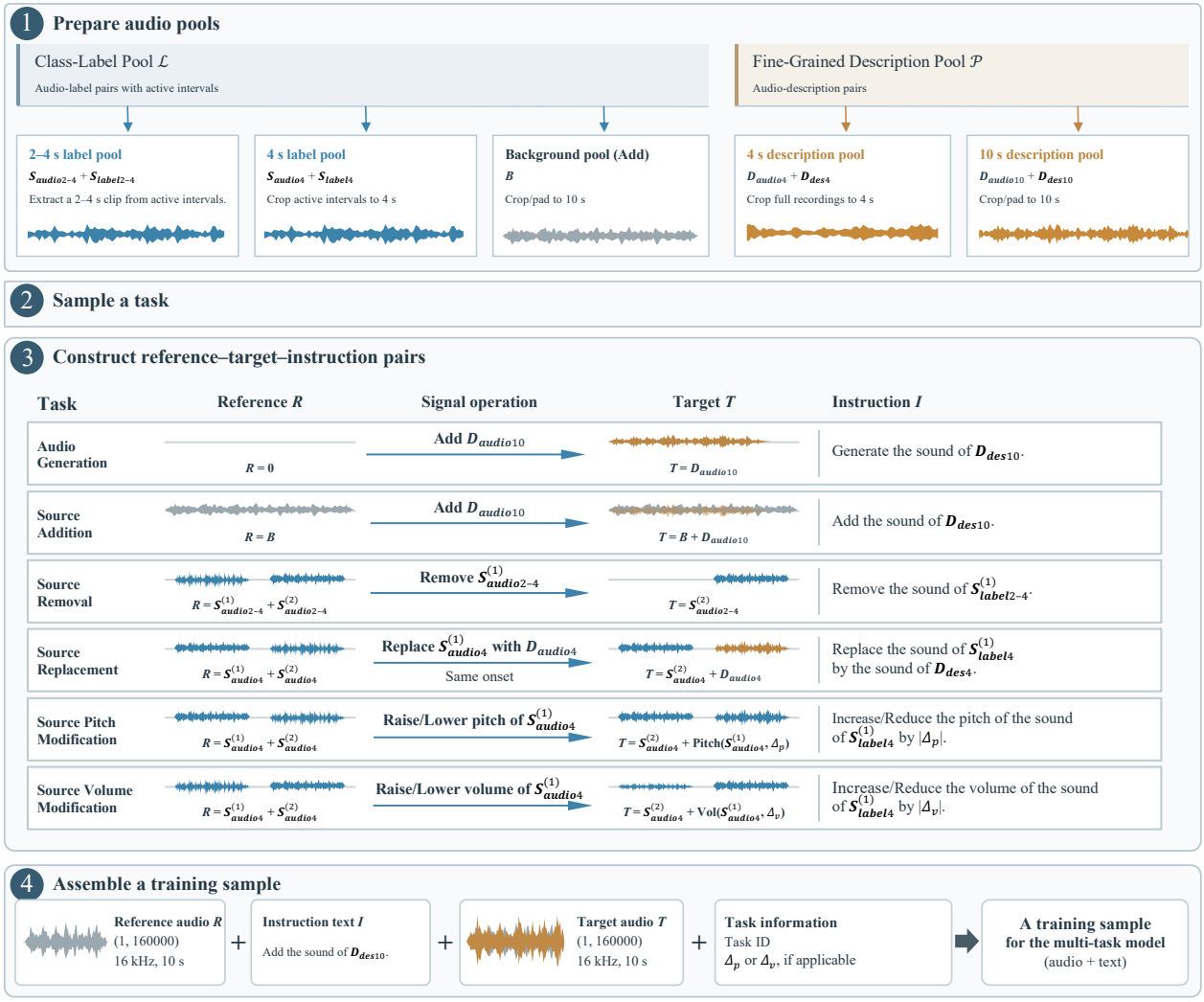


Figure 16 Overview of the Selective Audio Editing Benchmark constructor. Audio sources from the Class-Label Pool $\mathcal{L}$ and Fine-Grained Description Pool $\mathcal{P}$ are organized into task-specific pools and composed into reference–target–instruction pairs for six audio editing operations with controlled source layouts. $|\Delta_p|$ is fixed at 4 semitones, and $|\Delta_v|$ is fixed at 5 dB in this process.

In addition to the benchmark, we construct training and validation sets from the same controlled distribution to support controlled comparisons among different post-training strategies without introducing distribution differences. The training set contains 10,240 samples per task, resulting in 61,440 samples across the six tasks. The validation set and the benchmark each contain 1,000 samples per task. Throughout the construction process, we use the same audio source pools and the same source-level training/validation/test partition as the Complex Audio Editing data in Appendix B.2, ensuring that audio sources used for validation and benchmark evaluation do not appear in training.

D.2 Automatic Evaluation Protocol

The controlled source composition of the benchmark enables automatic evaluation of both editing success and selective editing failure types.

Task Success Identification and Selective Failure Categorization. Depending on the editing operation, we use CLAP similarity to measure source semantics, local RMS-level changes to measure removal and volume

Table 11 Automatic success criteria and selective failure categorization for the six editing tasks.

Task	Outcome	Target / edited source	Source to preserve
Audio Generation	**Success**	Target present (CLAP > 0.08)	–
	Other	Target absent (CLAP ≤ 0.08)	–
Source Addition	**Success**	Added source present (CLAP > 0.04)	–
	Other	Added source absent (CLAP ≤ 0.04)	–
Source Removal	**Success**	Removed (≥ 6 dB decrease)	Preserved ($\lvert\Delta v\rvert < 3$ dB)
	Target Missed	Not removed (< 6 dB decrease)	Preserved ($\lvert\Delta v\rvert < 3$ dB)
	Wrong-Target Edit	Not removed (< 6 dB decrease)	Removed (≥ 6 dB decrease)
	Unintended Non-Target Edit	Removed (≥ 6 dB decrease)	Removed (≥ 6 dB decrease)
	Other	Remaining cases	
Source Replacement	**Success**	Replacement source	Original preserved source
	Target Missed	Original target source	Original preserved source
	Wrong-Target Edit	Original target source	Replacement source
	Unintended Non-Target Edit	Replacement source	Replacement source
	Other	Ambiguous or unmatched cases	
Source Pitch Modification	**Success**	Modified ($r \in [0.8, 1.2]$)	Preserved ($\lvert\Delta p\rvert < 1$ semitone)
	Target Missed	Unchanged ($\lvert\Delta p\rvert < 1$ semitone)	Unchanged ($\lvert\Delta p\rvert < 1$ semitone)
	Wrong-Target Edit	Unchanged ($\lvert\Delta p\rvert < 1$ semitone)	Modified ($r \in [0.8, 1.2]$)
	Unintended Non-Target Edit	Modified ($r \in [0.8, 1.2]$)	Modified ($r \in [0.8, 1.2]$)
	Other	Remaining measurable cases	
Source Volume Modification	**Success**	Modified ($r \in [0.8, 1.2]$)	Preserved ($\lvert\Delta v\rvert < 1.5$ dB)
	Target Missed	Unchanged ($\lvert\Delta v\rvert < 1.5$ dB)	Unchanged ($\lvert\Delta v\rvert < 1.5$ dB)
	Wrong-Target Edit	Unchanged ($\lvert\Delta v\rvert < 1.5$ dB)	Modified ($r \in [0.8, 1.2]$)
	Unintended Non-Target Edit	Modified ($r \in [0.8, 1.2]$)	Modified ($r \in [0.8, 1.2]$)
	Other	Remaining cases	

For source replacement, a category is assigned only when its local CLAP similarity exceeds 0.08; if both candidate categories pass the threshold, the one with higher similarity is selected. For pitch and volume modification, Δp and Δv denote the measured pitch and volume changes, respectively, while r denotes the measured signed change divided by the requested signed change. Samples for which the pitch change cannot be measured for one or both sources are excluded from the final statistics.

modification, and multi-resolution CQT alignment to measure relative pitch changes. The success criterion and failure categorization for each task are defined as follows:

- **Audio Generation.** We compute the CLAP similarity between the generated audio and the target label. The generation is considered successful when the CLAP similarity is greater than 0.08; otherwise, the sample is categorized as *Other*.
- **Source Addition.** We compute the CLAP similarity between the edited audio and the label of the source to be added. The addition is considered successful when the CLAP similarity is greater than 0.04; otherwise, the sample is categorized as *Other*. The preservation of the original source is not included in the success criterion for source addition, as such failures are rare and additional preservation constraints can introduce false negatives.
- **Source Removal.** For source removal, the reference audio contains a source to be removed and a temporally non-overlapping source to be preserved. We measure the local volume changes of the two sources before and after editing. The edit is considered successful when the source to be removed decreases by at least 6 dB, while the absolute volume change of the source to be preserved remains below 3 dB. This design is also motivated by our observation that cases where the source to be preserved remains perceptually present but undergoes substantial waveform-level changes are rare. If neither source is removed, the result is categorized as *Target Missed*; if the source to be preserved is removed instead of the target source, it is categorized as *Wrong-Target Edit*; and if both sources are removed, it is categorized as *Unintended Non-Target Edit*. All remaining failures are categorized as *Other*.
- **Source Replacement.** For source replacement, the reference audio contains a source to be replaced and a temporally non-overlapping source to be preserved. We separately measure the local CLAP similarities within the two source regions. For the source to be replaced, we compare its original label with the replacement-source label; for the source to be preserved, we compare its original label with the replacement-source label. A source category is considered present only when its CLAP similarity is greater than 0.08, and the category with the higher similarity is selected when both satisfy the threshold. The edit is considered successful when the source to be replaced is identified as the replacement source while the source to be preserved retains its original identity. If both sources retain their original identities, the result is categorized as *Target Missed*; if the source to be replaced remains unchanged while the

source to be preserved is identified as the replacement source, it is categorized as *Wrong-Target Edit*; and if both regions are identified as the replacement source, it is categorized as *Unintended Non-Target Edit*. All ambiguous or otherwise unmatched cases are categorized as *Other*.

- **Source Pitch Modification.** For source pitch modification, the reference audio contains a source to be modified and a temporally non-overlapping source to be preserved. We separately estimate the relative pitch changes within the two source regions using multi-resolution CQT alignment. For the source to be modified, we compare its measured signed pitch change with the requested change; for the source to be preserved, we measure its pitch change relative to the original source. A source is considered successfully modified when its measured change reaches 80%–120% of the requested signed change, and is considered unchanged when its absolute pitch change is below 1 semitone. The edit is considered successful when the source to be modified is successfully modified while the source to be preserved remains unchanged. If both sources remain unchanged, the result is categorized as *Target Missed*; if the source to be modified remains unchanged while the source to be preserved undergoes the requested change, it is categorized as *Wrong-Target Edit*; and if both sources undergo the requested change, it is categorized as *Unintended Non-Target Edit*. All remaining failures are categorized as *Other*. Samples for which the pitch change cannot be measured for either source are excluded.
- **Source Volume Modification.** For source volume modification, the reference audio contains a source to be modified and a temporally non-overlapping source to be preserved. We separately measure the local volume changes within the two source regions. For the source to be modified, we compare its measured signed volume change with the requested change; for the source to be preserved, we measure its volume change relative to the original source. A source is considered successfully modified when its measured change reaches 80%–120% of the requested signed change, and is considered unchanged when its absolute volume change is below 1.5 dB. The edit is considered successful when the source to be modified is successfully modified while the source to be preserved remains unchanged. If both sources remain unchanged, the result is categorized as *Target Missed*; if the source to be modified remains unchanged while the source to be preserved undergoes the requested change, it is categorized as *Wrong-Target Edit*; and if both sources undergo the requested change, it is categorized as *Unintended Non-Target Edit*. All remaining failures are categorized as *Other*.

The criteria are summarized in Table 11. The task-level success rate and overall success rate are computed over valid samples.

E Details on the Effectiveness of STEER across Models and Datasets

This section provides the training configurations and learning dynamics of STEER across different models on the complex and selective audio editing datasets corresponding to Section 5.3.

E.1 STEER Adaptation on SAM Audio

Since SAM Audio adopts a different timestep formulation, we first describe how STEER is adapted to SAM Audio. Specifically, SAM Audio uses the opposite time parameterization from our base editor: $t = 0$ corresponds to noise, while $t = 1$ corresponds to clean audio. We therefore map the training strategy from the σ parameterization adopted by STEER to t through

$$t = 1 - \sigma.$$

Therefore, for STEER-Emphasis, the critical early range for the removal task, $\sigma \in [\sigma_2, 1]$, is correspondingly mapped to $t \in [0, 1 - \sigma_2]$. This preserves the same training semantics of emphasizing the high-noise, early-denoising region. For STEER-Recovery, we retain the 50-step noise-position grid and convert each position according to $t_j = 1 - \sigma_j$. Specifically, starting from the initial noise, we perform a k-step explicit Euler rollout using the conditional velocity field of the model:

$$\mathbf{z}_{j+1} = \mathbf{z}_j + (t_{j+1} - t_j)\mathbf{v}_\theta(\mathbf{z}_j, t_j, c).$$

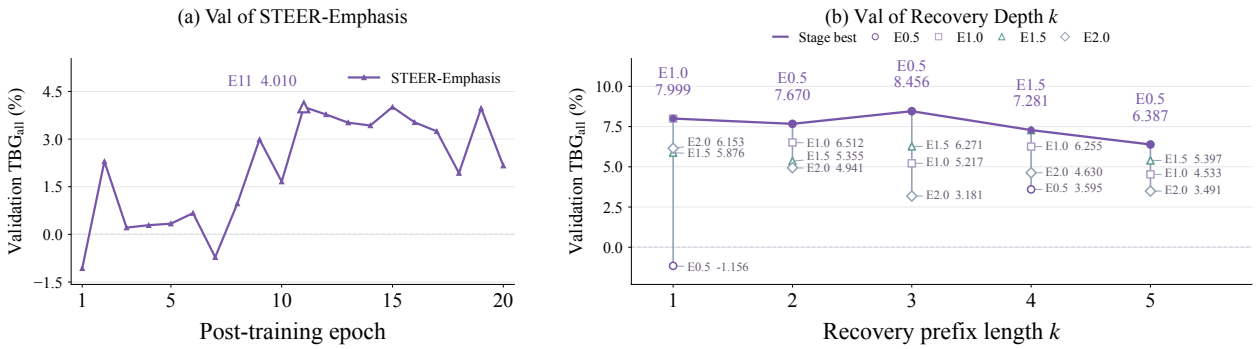


Figure 17 Validation dynamics of STEER on our base editor using the Complex Audio Editing validation set. (a) Validation TBG$_{all}$ across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Validation TBG$_{all}$ across recovery depths k. Markers denote checkpoints evaluated every 0.5 epoch at each depth, while the purple curve connects the validation-best checkpoint for each k.

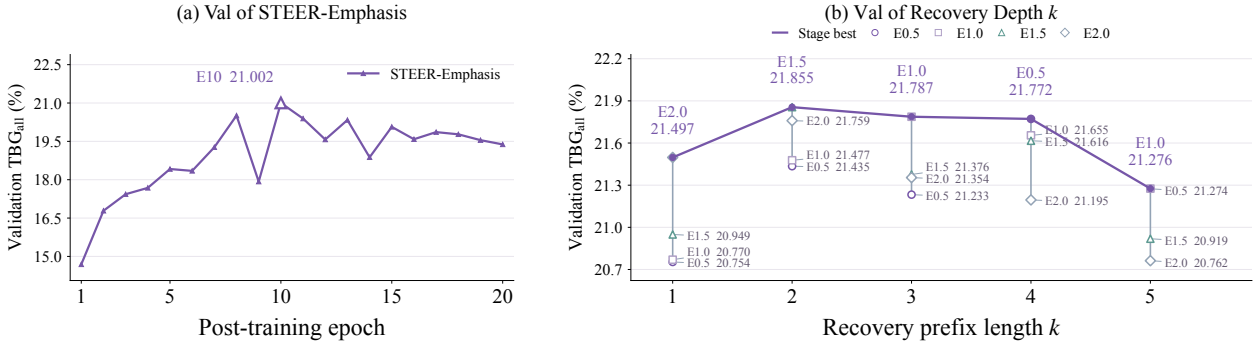


Figure 18 Validation dynamics of STEER on UNISON using the Complex Audio Editing validation set. (a) Validation TBG$_{all}$ across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Validation TBG$_{all}$ across recovery depths k. Markers denote checkpoints evaluated every 0.5 epoch at each depth, while the purple curve connects the validation-best checkpoint for each k.

After reaching the rollout state $\mathbf{z}_k$, we construct the corrective velocity toward the clean target latent as

$$\mathbf{v}_k^* = \frac{\mathbf{z}_{\mathrm{GT}} - \mathbf{z}_k}{1 - t_k},$$

where $\mathbf{z}_{\mathrm{GT}}$ denotes the clean latent obtained by concatenating the encoded target and residual tracks. The Euler rollout is used only to implement the STEER-Recovery post-training procedure and is not used for validation or inference. For all validation and test evaluations of SAM Audio, we follow its default midpoint ODE solver and integrate the trajectory using `torchdiffeq.odeint` with the default SAM Audio integration step size $\Delta t = \frac{2}{32} = 0.0625$. We also follow the default SAM Audio setting and do not use CFG during inference.

In addition, SAM Audio is a separation model that jointly outputs the target source and the residual audio. We post-train it only on the removal task, treating the source specified for removal in the editing instruction as the separation target and using the predicted residual audio as the final removal output. During training, both the target and residual branches are jointly supervised.

E.2 Training Details on The Complex Dataset across Models

We describe the training configurations and learning dynamics of STEER across different models on the complex audio editing datasets in this section.

For all three models, STEER-Emphasis starts from the corresponding pretrained checkpoint and runs for 20

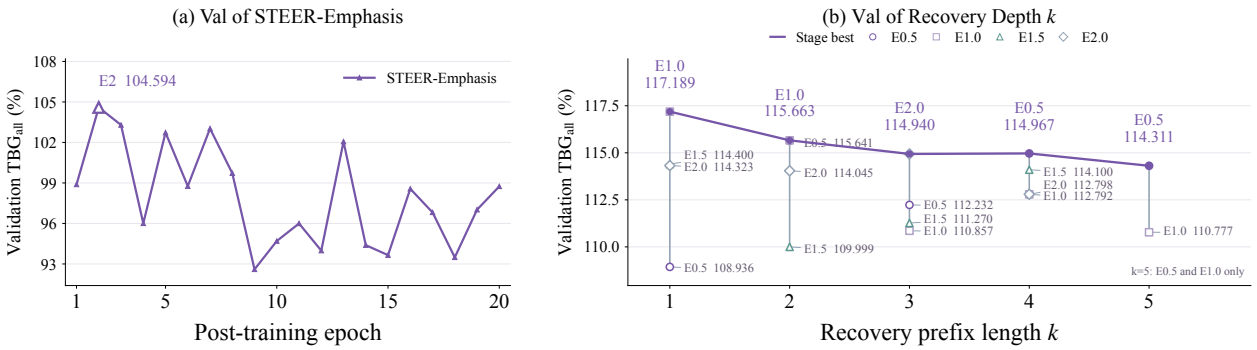


Figure 19 Validation dynamics of STEER on SAM Audio using the Complex Audio Editing validation set. (a) Validation TBG_{all} across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Validation TBG_{all} across recovery depths k. Markers denote checkpoints evaluated every 0.5 epoch at each depth, while the purple curve connects the validation-best checkpoint for each k.

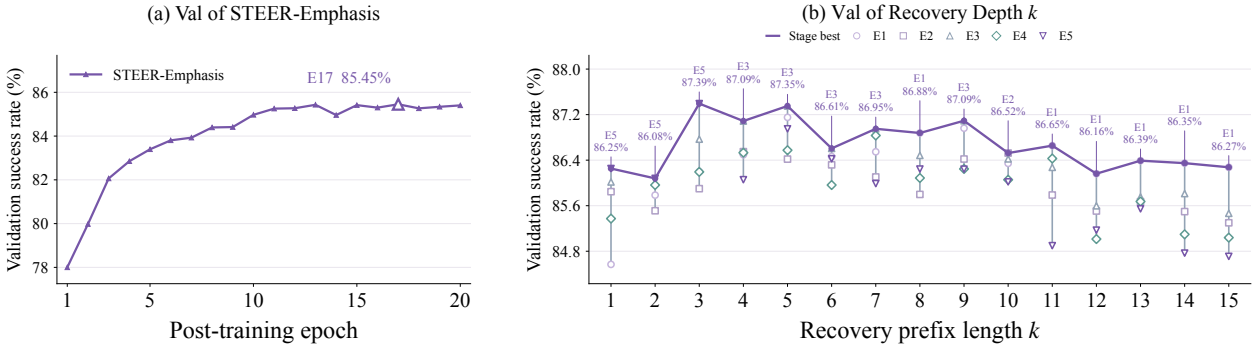


Figure 20 Validation dynamics of STEER on our base editor using the Selective Audio Editing validation set. (a) Overall validation success rate across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Overall validation success rate across recovery depths k. Markers denote checkpoints from the five training epochs at each depth, while the purple curve connects the validation-best checkpoint for each k.

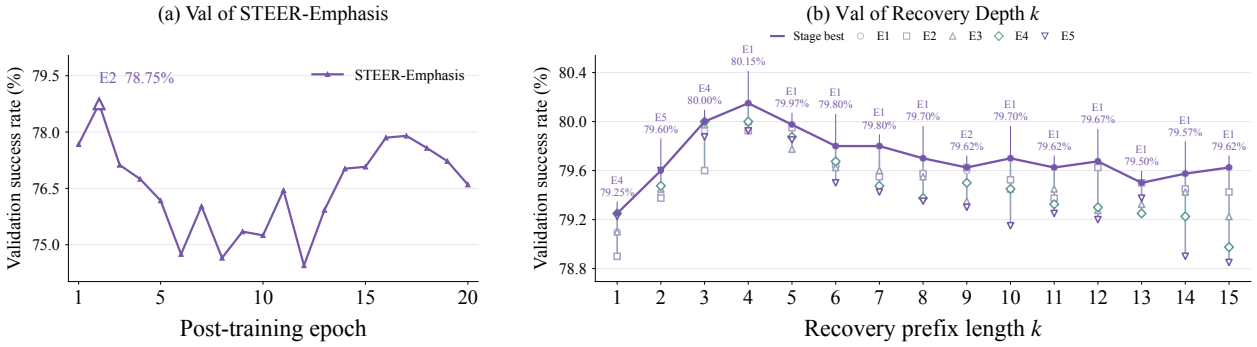


Figure 21 Validation dynamics of STEER on UNISON using the Selective Audio Editing validation set. (a) Overall validation success rate across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Overall validation success rate across recovery depths k. Markers denote checkpoints from the five training epochs at each depth, while the purple curve connects the validation-best checkpoint for each k.

epochs. We select the checkpoint with the highest validation TBG_{all}, which summarizes overall performance across the tasks supported by each model relative to its pretrained baseline. STEER-Recovery starts from the selected STEER-Emphasis checkpoint and is trained for 2 epochs at each recovery depth k, with validation performed every 0.5 epoch. At each subsequent depth, training starts from the best checkpoint found so

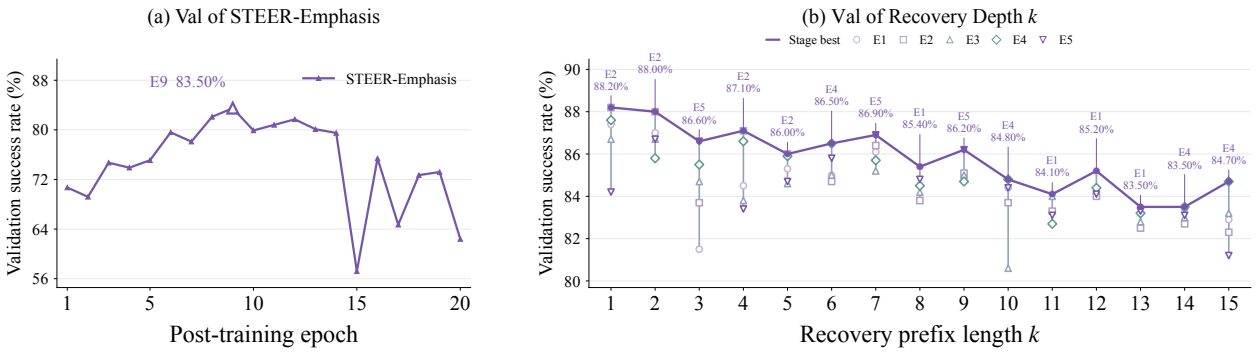


Figure 22 Validation dynamics of STEER on SAM Audio using the Selective Audio Editing validation set. (a) Overall validation success rate across STEER-Emphasis training epochs, with the selected checkpoint highlighted. (b) Overall validation success rate across recovery depths k. Markers denote checkpoints from the five training epochs at each depth, while the purple curve connects the validation-best checkpoint for each k.

far according to validation $\mathrm{TBG}_{\mathrm{all}}$, with the optimizer state reinitialized. We set the patience parameter to $P = 2$. For SAM Audio, we follow the model adaptation procedure described in Section E.1. Across all models, we train only the DiT backbone, keeping all other components, including the language encoders and audio VAEs or codecs, frozen.

To account for differences in model size and pretrained editing capabilities, we use model-specific learning rates and batch sizes. Specifically, our base model uses learning rates of 2×10^{-6} for STEER-Emphasis and 1×10^{-5} for STEER-Recovery, with a global batch size of 480 in both stages. UNISON uses learning rates of 1×10^{-5} and 1×10^{-6} for the two stages, respectively, with a global batch size of 128 throughout. For SAM Audio, we use the Small variant with a learning rate of 1×10^{-4} and a global batch size of 1,024 in both stages. The validation curves for different models on the complex validation set are shown in Figures 17, 18, and 19, respectively. And the final test results are shown in Table 2

E.3 Training Details on The Selective Dataset across Models

For all three models, STEER-Emphasis starts from the corresponding pretrained checkpoint and runs for 20 epochs. We select the checkpoint with the highest overall validation success rate, which summarizes editing performance across the tasks supported by each model. STEER-Recovery starts from the selected STEER-Emphasis checkpoint and is trained for 5 epochs at each recovery depth k, with validation performed once per epoch. At each subsequent depth, training starts from the best checkpoint found so far according to the overall validation success rate, with the optimizer state reinitialized. We set the patience parameter to $P = 2$. For SAM Audio, we follow the model adaptation procedure described in Section E.1. Across all models, we train only the DiT backbone, keeping all other components, including the language encoders and audio VAEs or codecs, frozen.

To account for differences in model size and pretrained editing capabilities, we use model-specific learning rates and batch sizes. Specifically, our base model uses peak learning rates of 2×10^{-6} for STEER-Emphasis and 1×10^{-5} for STEER-Recovery, with a global batch size of 480 in both stages. UNISON uses learning rate of 1×10^{-5} and a global batch size of 32 in both stages. For SAM Audio, we use the Small variant with a learning rate of 1×10^{-4} and a global batch size of 1,024 in both stages. The validation curves for different models on the selective validation set are shown in Figures 20, 21, and 22, respectively. And the final test results are shown in Table 3

Additionally, on the selective dataset, we continue STEER-Recovery training beyond the stopping point prescribed by $P = 2$, up to $k = 15$, to examine whether deeper recovery yields further gains. Across different models, none of the subsequently evaluated depths surpasses the previous validation best, further supporting the effectiveness of the patience-based stopping rule.

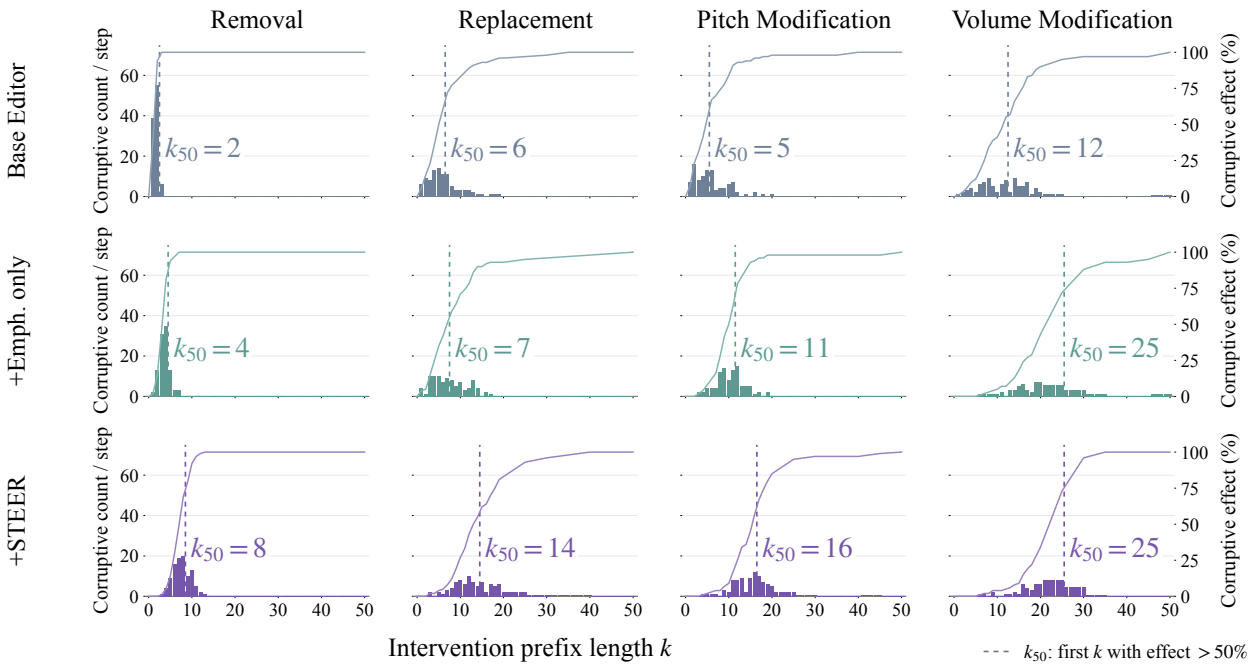


Figure 23 Task-wise recovery from early-trajectory corruption across model variants on the Selective Audio Editing Benchmark. Larger k_{50} indicates stronger robustness to early-trajectory errors.



Figure 24 Human evaluation of our base editor, its STEER-Emphasis-only variant, and full STEER variant on the Complex Audio Editing Test Set, using 100 randomly sampled examples per task. Left: overall success rates and selective error counts. Right: task-wise distributions of successful edits and different failure types.

F Details on Analysis of Source-Selection Reliability

This section provides additional results supporting the source-selection reliability analysis in Section 5.4. Specifically, we present task-wise evaluations of recovery from early-trajectory errors and detailed human evaluation results across different editing tasks.

F.1 Additional Results on Recovery from Early-Trajectory Errors

Figure 23 provides the task-wise results corresponding to Figure 8. Across all four selective editing tasks, STEER-Emphasis improves robustness to early-trajectory corruption. STEER-Recovery further increases k_{50} substantially for removal, replacement, and pitch modification, while maintaining the gain on volume modification. These results further confirm that STEER-Recovery improves recovery from early-trajectory errors across selective editing tasks

F.2 Additional results on human evaluation.

Figure 24 provides task-wise results corresponding to Figure 9. The task-wise results provide a more detailed view of these improvements across different variants.

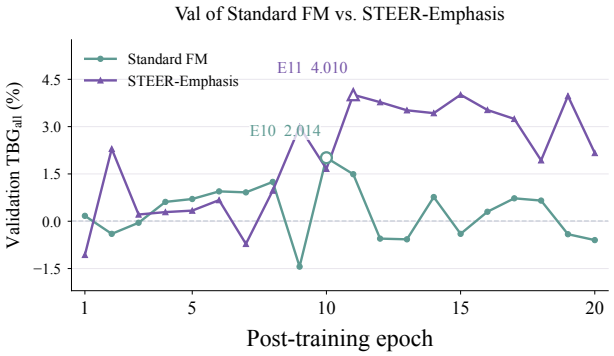


Figure 25 Validation dynamics of standard Flow Matching and STEER-Emphasis on our base editor using the Complex Audio Editing validation set. Curves show validation TBG_{all} across post-training epochs, with the validation-best checkpoint of each method highlighted.

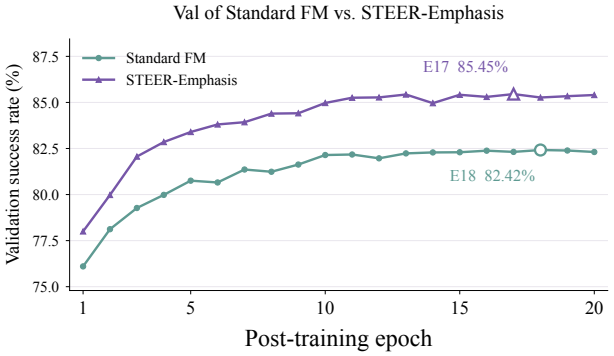


Figure 26 Validation dynamics of standard Flow Matching and STEER-Emphasis on our base editor using the Selective Audio Editing validation set. Curves show overall validation success rates across post-training epochs, with the validation-best checkpoint of each method highlighted.

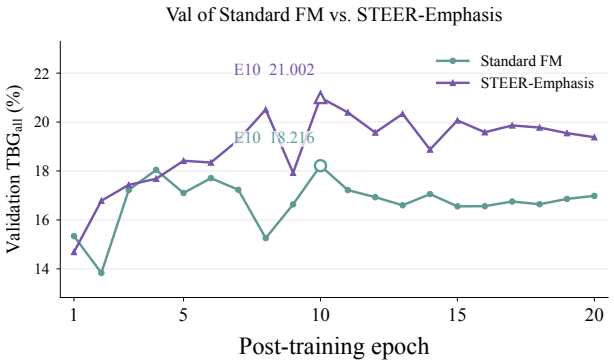


Figure 27 Validation dynamics of standard Flow Matching and STEER-Emphasis on UNISON using the Complex Audio Editing validation set. Curves show validation TBG_{all} across post-training epochs, with the validation-best checkpoint of each method highlighted.

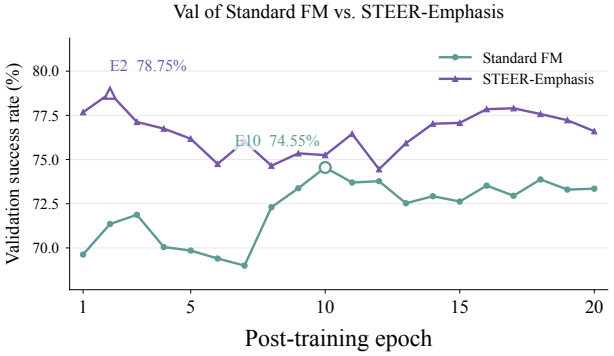


Figure 28 Validation dynamics of standard Flow Matching and STEER-Emphasis on UNISON using the Selective Audio Editing validation set. Curves show overall validation success rates across post-training epochs, with the validation-best checkpoint of each method highlighted.

Table 12 Detailed comparison of STEER-Emphasis and standard Flow Matching post-training under matched updates on the Complex Audio Editing Test Set. TBG scores are computed relative to the corresponding pretrained model.

Model	Strategy	Generate		Add		Replace		Remove			Pitch			Volume			Overall	
		CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	LSD↓	KL↓	MAE↓	LSD↓	KL↓	MAE↓	LSD↓	KL↓	TBG_{all} ↑	TBG_{sel} ↑
Our Base Editor	Standard FM	**0.23**	**1.99**	**0.25**	**1.31**	**0.24**	1.08	**0.14**	1.33	1.06	0.05	0.49	**0.27**	0.73	**0.43**	**0.20**	0.92	0.66
	STEER-Emphasis	0.22	2.12	0.25	1.33	0.23	**1.05**	0.14	**1.22**	**0.90**	**0.05**	**0.49**	0.28	**0.71**	**0.43**	0.20	**1.67**	**2.97**
UNISON	Standard FM	0.24	1.87	0.26	**1.26**	0.23	1.24	0.14	1.49	1.17	–	–	–	–	–	–	6.97	12.89
	STEER-Emphasis	**0.25**	**1.84**	**0.26**	1.37	**0.24**	**1.16**	**0.15**	**1.39**	**0.89**	–	–	–	–	–	–	**10.27**	**17.85**
SAM Audio	Standard FM	–	–	–	–	–	–	0.10	1.47	1.45	–	–	–	–	–	–	65.69	65.69
	STEER-Emphasis	–	–	–	–	–	–	**0.12**	**1.43**	**1.14**	–	–	–	–	–	–	**108.62**	**108.62**

G Details on Ablation Studies

This subsection provides the training configurations, learning dynamics and additional results for the ablation studies presented in Section 5.5.

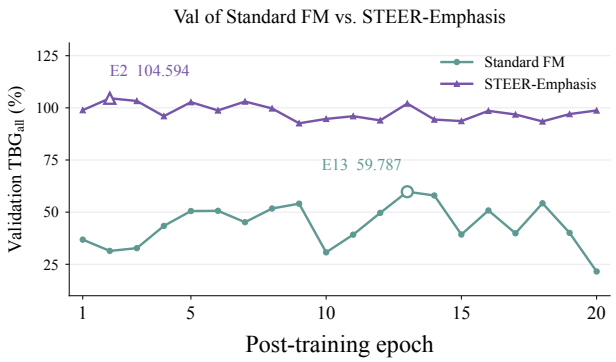


Figure 29 Validation dynamics of standard Flow Matching and STEER-Emphasis on SAM Audio using the Complex Audio Editing validation set. Curves show validation TBG_{all} across post-training epochs, with the validation-best checkpoint of each method highlighted.

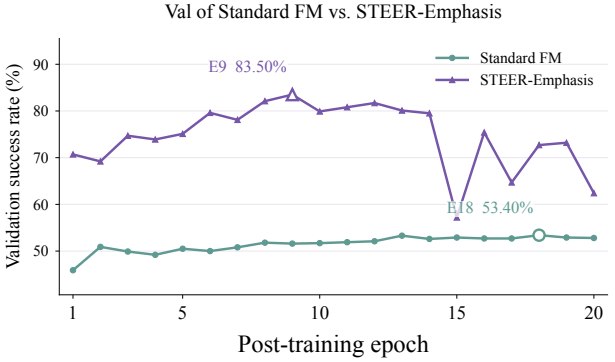


Figure 30 Validation dynamics of standard Flow Matching and STEER-Emphasis on SAM Audio using the Selective Audio Editing validation set. Curves show validation removal success rates across post-training epochs, with the validation-best checkpoint of each method highlighted.

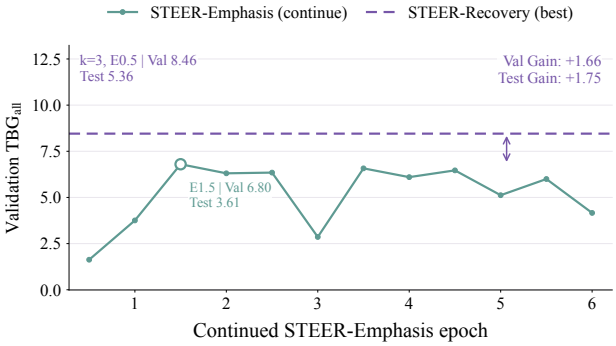


Figure 31 Comparison of continued STEER-Emphasis and STEER-Recovery on our base editor using the Complex Audio Editing validation set under matched optimizer updates. The solid curve shows validation TBG_{all} during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

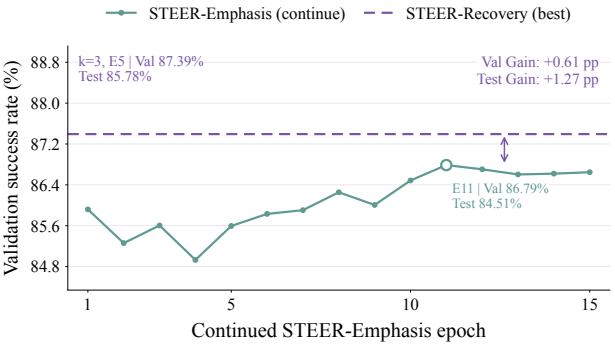


Figure 32 Comparison of continued STEER-Emphasis and STEER-Recovery on our base editor using the Selective Audio Editing validation set under matched optimizer updates. The solid curve shows overall validation success rates during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

G.1 Training Details for the STEER-Emphasis Ablation

We compare STEER-Emphasis with standard Flow Matching post-training under matched numbers of optimizer updates on different datasets across different models. Within each model–dataset pair, both runs start from the same pretrained checkpoint and use the same training data, trainable components, batch size, and optimization settings as STEER-Emphasis described in Section E.2 and E.3. The standard Flow Matching baseline uses logit-normal noise-level sampling for all supported tasks, whereas STEER-Emphasis adopts the task-dependent sampling distributions in Equation 3. The corresponding validation performance curves for the two strategies are shown in Figure 25, 26, 27, 28, 29 and 30. Detailed task-wise results corresponding to Table 4 are provided in Table 12 and 13.

G.2 Training Details for the STEER-Recovery Ablation

We compare STEER-Recovery with continued STEER-Emphasis training under matched numbers of optimizer updates. The continuation baseline, denoted $\text{STEER-Emphasis}_{\text{cont}}$, retains the STEER-Emphasis noise-level

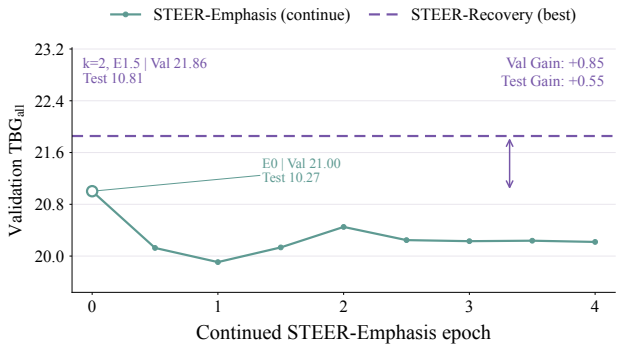


Figure 33 Comparison of continued STEER-Emphasis and STEER-Recovery on UNISON using the Complex Audio Editing validation set under matched optimizer updates. The solid curve shows validation TBG$_{all}$ during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

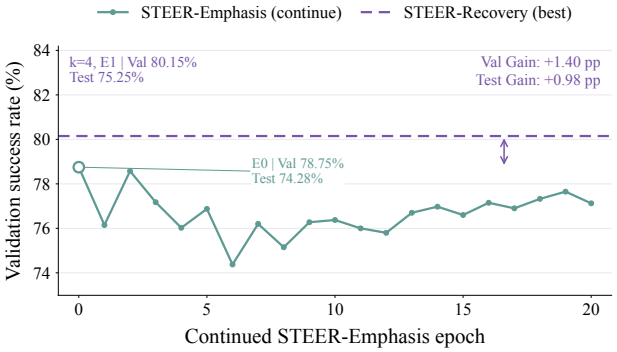


Figure 34 Comparison of continued STEER-Emphasis and STEER-Recovery on UNISON using the Selective Audio Editing validation set under matched optimizer updates. The solid curve shows overall validation success rates during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

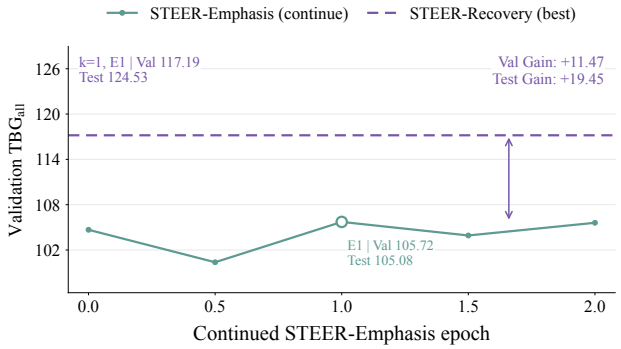


Figure 35 Comparison of continued STEER-Emphasis and STEER-Recovery on SAM Audio using the Complex Audio Editing validation set under matched optimizer updates. The solid curve shows validation TBG$_{all}$ during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

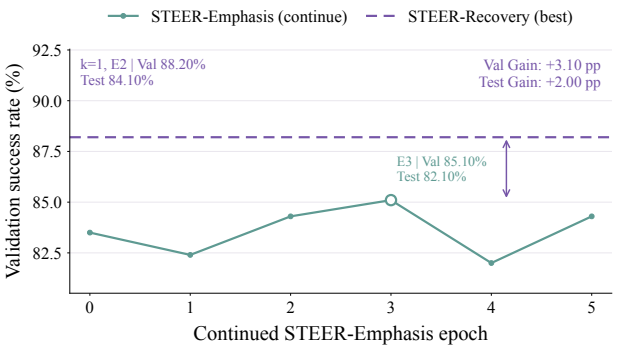


Figure 36 Comparison of continued STEER-Emphasis and STEER-Recovery on SAM Audio using the Selective Audio Editing validation set under matched optimizer updates. The solid curve shows validation removal success rates during continued STEER-Emphasis training, while the dashed line indicates the validation performance of the selected STEER-Recovery checkpoint. The highlighted marker denotes the validation-best continuation checkpoint. Validation and test gains are annotated.

sampling distribution and the standard Flow Matching objective, rather than introducing recovery supervision on self-generated trajectory states.

For a fair comparison, both strategies are initialized from the same validation-best STEER-Emphasis checkpoint, ensuring identical starting model weights. STEER-Emphasis$_{cont}$ uses the same peak learning rate and global batch size as the corresponding STEER-Recovery run. However, since resetting the optimizer during continuous fine-tuning would disrupt optimization and is not part of standard training practice, we retain the optimizer state throughout continuation training, without checkpoint rollback or optimizer restarts. We also follow the same validation frequency as STEER-Recovery described in Sections E.2 and E.3, evaluating every 0.5 epoch on the complex dataset and once per epoch on the selective dataset.

Table 13 Detailed comparison of STEER-Emphasis and standard Flow Matching post-training under matched updates on the Selective Audio Editing Benchmark. Succ. and SErr. denote success and selective error rates, respectively.

Model	Strategy	Generate	Add	Replace		Remove		Pitch		Volume		Overall	
		Succ.↑	Succ.↑	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓
Our Base Editor	Standard FM	93.70	92.90	76.20	16.40	83.00	13.40	82.44	16.91	58.40	8.40	81.05	13.58
	STEER-Emphasis	**94.00**	**94.60**	**78.90**	**14.00**	**88.10**	**7.90**	**87.90**	**11.70**	**61.60**	**6.80**	**84.02**	**9.99**
UNISON	Standard FM	**95.20**	82.40	67.40	23.30	27.70	14.60	–	–	–	–	68.18	18.95
	STEER-Emphasis	94.80	**85.60**	**78.40**	**13.60**	**38.30**	**8.50**	–	–	–	–	**74.28**	**11.05**
SAM Audio	Standard FM	–	–	–	–	50.50	49.40	–	–	–	–	50.50	49.40
	STEER-Emphasis	–	–	–	–	**80.60**	**19.10**	–	–	–	–	**80.60**	**19.10**

Table 14 Detailed comparison of STEER-Recovery and continued STEER-Emphasis under matched updates on the Complex Audio Editing Test Set. TBG scores are computed relative to the corresponding pretrained model.

Model	Strategy	Generate		Add		Replace		Remove			Pitch			Volume			Overall	
		CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	LSD↓	KL↓	MAE↓	LSD↓	KL↓	MAE↓	LSD↓	KL↓	TBG_{all} ↑	TBG_{sel} ↑
Our Base Editor	Emphasis (continue)	0.22	**2.08**	**0.25**	**1.32**	**0.24**	1.04	0.14	1.25	0.93	**0.05**	**0.49**	**0.28**	0.68	**0.43**	**0.20**	3.61	5.50
	STEER-Recovery	**0.22**	2.13	0.25	1.35	**0.24**	**1.03**	**0.14**	**1.16**	**0.79**	**0.05**	**0.49**	0.28	**0.63**	**0.43**	**0.20**	**5.36**	**8.25**
UNISON	Emphasis (continue)	0.25	1.84	0.26	1.37	**0.24**	1.16	0.15	1.39	0.89	–	–	–	–	–	–	10.27	17.85
	STEER-Recovery	**0.25**	**1.83**	**0.27**	**1.32**	0.23	**1.16**	**0.15**	**1.36**	**0.82**	–	–	–	–	–	–	**10.81**	**18.42**
SAM Audio	Emphasis (continue)	–	–	–	–	–	–	0.12	1.36	1.06	–	–	–	–	–	–	105.08	105.08
	STEER-Recovery	–	–	–	–	–	–	**0.13**	**1.27**	**0.97**	–	–	–	–	–	–	**124.53**	**124.53**

Table 15 Detailed comparison of STEER-Recovery and continued STEER-Emphasis under matched updates on the Selective Audio Editing Benchmark. Succ. and SErr. denote success and selective error rates, respectively.

Model	Strategy	Generate	Add	Replace		Remove		Pitch		Volume		Overall	
		Succ.↑	Succ.↑	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓
Our Base Editor	Emphasis (continue)	93.90	93.20	**80.70**	12.60	**88.30**	8.00	91.05	8.55	61.50	**5.50**	84.51	8.67
	STEER-Recovery	**94.80**	**93.50**	79.10	**12.30**	88.20	**7.90**	**92.83**	**6.77**	**68.00**	6.40	**85.78**	**8.45**
UNISON	Emphasis (continue)	94.80	85.60	78.40	13.60	38.30	8.50	–	–	–	–	74.28	11.05
	STEER-Recovery	**94.90**	85.40	**79.60**	**13.10**	**41.10**	**8.00**	–	–	–	–	**75.25**	**10.55**
SAM Audio	Emphasis (continue)	–	–	–	–	82.10	17.50	–	–	–	–	82.10	17.50
	STEER-Recovery	–	–	–	–	**84.10**	**15.70**	–	–	–	–	**84.10**	**15.70**

Across all models and datasets, we match the continuation training budget to the total number of optimizer updates performed through completion of the recovery depth containing the selected STEER-Recovery checkpoint. Specifically, for our base editor, we continue training for 6 epochs on the complex dataset and 15 epochs on the selective dataset. For UNISON, we continue training for 4 and 20 epochs, respectively. For SAM Audio, we continue training for 2 and 5 epochs, respectively.

The validation trajectories of continued STEER-Emphasis across training epochs are shown in Figures 31, 32, 33, 34, 35 and 36. Detailed task-wise results corresponding to Table 5 are provided in Tables 14 and 15.

G.3 Training Details for the Alternative Early-Trajectory Range Selection.

We conduct this ablation on our base editor in both dataset settings. The alternative variant, STEER-Emphasis$_{\text{elbow}}$, replaces the task-group-specific ranges defined by the cumulative-effect threshold $\rho = 0.8$ with a shared early-trajectory range. Specifically, we apply an elbow-based rule to the cumulative corrective-effect curve averaged across the four selective editing tasks, obtaining a shared range covering the first 15 of 50 denoising steps, as shown in Figure 10. Accordingly, STEER-Emphasis$_{\text{elbow}}$ samples σ uniformly from $[\sigma_{15}, 1]$ for removal, replacement, pitch modification, and volume modification, while generation and addition retain logit-normal sampling. Both variants start from the same base-editor checkpoint and follow the same dataset-specific STEER-Emphasis training configurations as described in Section E.2 and E.3. Detailed task-wise results corresponding to Table 6 are provided in Table 16 and 17.

Table 16 Task-wise comparison of task-group-specific and elbow-based shared early-trajectory ranges for STEER-Emphasis on the Complex Audio Editing Test Set. TBG scores are computed relative to our base editor.

Dataset	Strategy	Generate		Add		Replace		Remove			Pitch			Volume			Overall	
		CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	KL↓	CLAP↑	LSD↓	KL↓	MAE↓	LSD↓	KL↓	MAE↓	LSD↓	KL↓	TBG_{all} ↑	TBG_{sel} ↑
Complex	Emph. (elbow)	0.22	2.14	0.25	1.35	0.23	1.06	**0.14**	1.25	0.95	**0.05**	**0.49**	**0.27**	0.74	**0.43**	**0.20**	1.36	2.82
	STEER-Emphasis	**0.22**	**2.12**	**0.25**	**1.32**	**0.23**	**1.05**	0.14	**1.22**	**0.90**	0.05	0.49	0.28	**0.71**	0.43	0.20	**1.67**	**2.97**

Table 17 Task-wise comparison of task-group-specific and elbow-based shared early-trajectory ranges for STEER-Emphasis on the Selective Audio Editing Benchmark. Succ. and SErr. denote success and selective error rates, respectively.

Dataset	Strategy	Generate	Add	Replace		Remove		Pitch		Volume		Overall	
		Succ.↑	Succ.↑	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓	Succ.↑	SErr.↓
Selective	Emph. (elbow)	93.80	94.10	**79.80**	**13.80**	87.80	8.40	**89.33**	**10.13**	60.40	**6.50**	83.98	**9.68**
	STEER-Emphasis	**94.00**	**94.60**	78.90	14.00	**88.10**	**7.90**	87.90	11.70	**61.60**	6.80	**84.02**	9.99

H Details on Extension to Fine-Grained Selective Control

To complement the experiments in Section 5.6, this appendix provides additional experimental details and extended results on two fine-grained selective-control tasks: transcript-conditioned target speaker removal (TSR) with our base editor and target speaker separation (TSS) with SAM Audio. Unlike category-level audio editing, both tasks require identifying the target speaker based on spoken content. Appendix H.1 and Appendix H.2 present the respective task settings, model adaptations, training configurations, evaluation metrics, and experimental results.

H.1 Details on Target Speaker Removal with Our Base Editor

In this section, we first introduce the TSR task setting, followed by the model adaptation and training configuration. Finally, we analyze the corresponding training results.

Task Setting. Existing audio editing models and tasks often specify the editing target at the category level, e.g., "Remove the sound of [class label]" from a mixture containing sources from different categories. This is also the setting adopted by our base editor, where the target source is specified by a class label. In many practical scenarios, however, the target must be identified from substantially finer-grained conditions. To investigate whether early-trajectory emphasis also benefits fine-grained selective control, we extend our base editor to transcript-conditioned *Target Speaker Removal (TSR)*, where the model identifies and removes a target speaker based on spoken content while preserving the remaining speech. Given a speech mixture

$$x = s_{\text{target}} + s_{\text{residual}}, \tag{11}$$

where s_{target} denotes the speech signal of the target speaker to be removed and s_{residual} denotes the remaining speech that should be preserved, the model learns

$$\hat{s}_{\text{residual}} = f_\theta(x, c), \tag{12}$$

rather than generating s_{target}. The condition c is constructed from the transcription of the target speaker. During training, the instruction takes the form

$$\texttt{\{Mix audio\} Remove the sound of \{transcription\}}, \tag{13}$$

where `{Mix audio}` denotes the *input mixed audio*, while `{transcription}` denotes the *text transcription* of the target speaker's speech to be removed. For example, if the target speaker says "Please close the door," the corresponding instruction is "{Mix audio} Remove the sound of Please close the door." The model must therefore use the textual content to locate the corresponding speaker within the mixture, remove the sound of that speaker, and preserve the remaining speech.

Model Adaptation, Training Strategy, and Trainable Parameters.

Since this task requires the model to understand the linguistic content of the input audio, we replace the original T5 text encoder in our base editor with a multimodal large language model (MLLM) and connect the MLLM to the DiT backbone of our base editor through a Meta Query [21] mechanism. Specifically, we use MOSS-Audio-4B-Thinking [34] as the audio-content understanding module. MOSS jointly receives the mixed audio, the text removal instruction, and 50 learnable queries inserted after the `<SEG>` token. We extract the corresponding query hidden states before the output head of MOSS and pass them through a one-layer Transformer Connector followed by a LayerNorm–MLP Projector. The resulting representations are mapped into conditioning features and replace the original T5-based textual conditioning of our base editor.

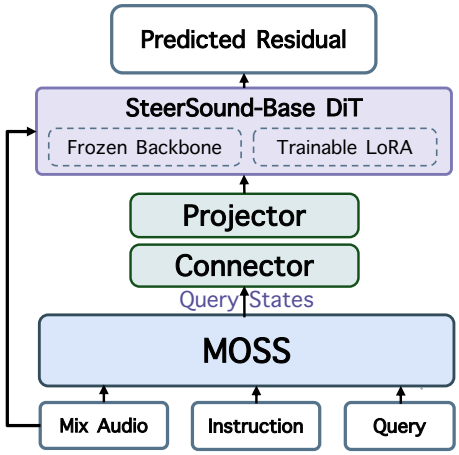


Figure 37 Architecture of our base editor adapted for the TSR task.

This adaptation introduces newly initialized components, including the learnable queries, Transformer Connector, and Projector. Moreover, we observe that standard Flow Matching training barely converges on this fine-grained task and fails to learn effective target selection. Therefore, rather than first training these newly introduced components with standard Flow Matching and subsequently applying STEER-Emphasis as a post-training stage, we directly compare standard training with training that incorporates the early-trajectory emphasis principle from the beginning. This setting allows us to more clearly assess the effect of emphasizing the early trajectory throughout training. The architecture of our base editor adapted for the TSR task is shown in Figure 37.

Since fine-grained speaker editing differs substantially from the original audio editing tasks, we adapt the DiT backbone of our base editor with LoRA to avoid disrupting its previously learned editing capability. Specifically, LoRA modules are inserted into the `qkv` and `linear` layers of the attention blocks, with rank 64, alpha 128, and dropout 0.05. We jointly train the DiT LoRA parameters, learnable queries, Transformer Connector, and Projector, while keeping all other components frozen. The numbers of trainable parameters for different modules are summarized in Table 18.

Table 18 Trainable parameters.

Module	Params.
DiT LoRA	15.73M
Queries	0.13M
Connector	52.46M
Projector	8.53M
Total	**76.84M**

Training Configuration. We initialize the model from our base editor and train it for 20 epochs using either Standard FM or the early-emphasis training strategy. Both settings use AdamW with a weight decay of 0.01, a constant learning rate of 1×10^{-4}, no warmup or learning-rate decay, a batch size of 64, and gradient clipping with a maximum norm of 1.0.

Since several trainable components, including the DiT LoRA, Connector, and Projector, are newly initialized, this setting does not fully satisfy the original assumption of STEER-Emphasis that the model already possesses an initial editing capability. We therefore adopt a mixed sampling strategy: while most training states follow the STEER-Emphasis σ sampling, a small proportion are sampled using the standard logit-normal distribution to retain coverage of the full generation trajectory. Specifically, Standard FM uses the standard logit-normal sampling strategy $\sigma \sim \mathrm{sigmoid}\big(\mathcal{N}(0,1)\big)$. For the early-emphasis training, we use

$$\sigma \sim \begin{cases} \mathcal{U}(0.97, 1), & \text{with probability } 0.8, \\ \mathrm{sigmoid}\big(\mathcal{N}(0,1)\big), & \text{with probability } 0.2. \end{cases}$$

Thus, 80% of the training states are concentrated in the high-noise early-trajectory region, while the remaining 20% retain standard logit-normal sampling to preserve coverage of the full generation trajectory. The training objective remains the standard Flow Matching objective in both settings.

Training Dataset. We use the PromptTSE [27] dataset for training and evaluation. In PromptTSE, each mixed-audio sample contains speech from two distinct speakers and is accompanied by the target-speaker audio to be extracted, the residual speech after removing the target speaker, and the transcription of the

target speaker. In our TSR setting, the provided residual audio is used as the generation target, while the target transcription is used to construct the fine-grained removal instruction.

Evaluation Metric. We directly use CLAP Audio–Audio similarity for TSR (CLAP-AA$_{\text{TSR}}$) to measure the similarity between the generated residual audio and the ground-truth residual audio. Let $E_a(\cdot)$ denote the CLAP [5] audio encoder. For sample i,

$$\text{CLAP-AA}_{\text{TSR},i} = \frac{E_a(\hat{s}_{\text{residual},i})^\top E_a(s_{\text{residual},i})}{\|E_a(\hat{s}_{\text{residual},i})\|_2 \, \|E_a(s_{\text{residual},i})\|_2}.$$

The final score is averaged over all N evaluation samples:

$$\text{CLAP-AA}_{\text{TSR}} = \frac{1}{N} \sum_{i=1}^{N} \text{CLAP-AA}_{\text{TSR},i}\,.$$

CLAP-AA$_{\text{TSR}}$ therefore directly measures the audio similarity between the generated residual audio and the ground-truth residual audio.

Experimental Results. Figure 38 shows the validation CLAP-AA$_{\text{TSR}}$ of Standard FM and early trajectory emphasis training over 20 training epochs. Compared with Standard FM, early trajectory emphasis training learns substantially more effectively. This result further demonstrates the importance of emphasizing the training on critical early-trajectory states for fine-grained selective control task.

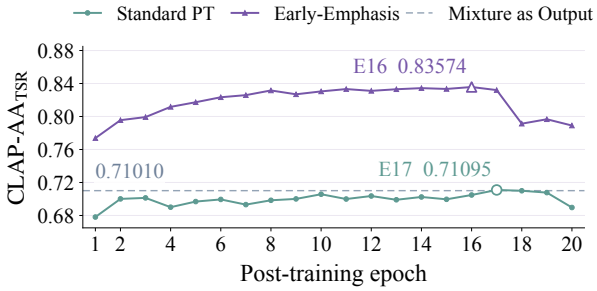


Figure 38 Validation CLAP-AA$_{\text{TSR}}$ during training on the TSR task with our adapted base editor. The horizontal line denotes the CLAP-AA$_{\text{TSR}}$ of the mixture audio.

The horizontal line denotes the CLAP-AA$_{\text{TSR}}$ obtained by directly using the mixed audio as the model output and comparing it with the ground-truth residual audio. Notably, without emphasizing the training on critical early trajectory, Standard FM performs close to the direct-mixture baseline, whereas emphasizing the training on critical early trajectory substantially improves performance on this fine-grained selective control task.

H.2 Details on Target Speaker Separation with SAM Audio

In this section, we first introduce the TSS task setting, followed by the model adaptation and training configuration. Finally, we analyze the corresponding training results.

Task Setting. Following the experiment with our base editor, we further investigate whether early-trajectory emphasis benefits fine-grained selective control on SAM Audio. We consider transcript-conditioned *Target Speaker Separation (TSS)*, in which the model identifies the target speaker based on spoken content and separates the mixture into target and residual speech. Given a speech mixture

$$x = s_{\text{target}} + s_{\text{residual}}, \tag{14}$$

where s_{target} denotes the speech signal of the target speaker to be separated and s_{residual} denotes the remaining speech, the model learns

$$[\hat{s}_{\text{target}}, \hat{s}_{\text{residual}}] = f_\theta(x, c), \tag{15}$$

with supervision applied to both $\hat{s}_{\text{target}}$ and $\hat{s}_{\text{residual}}$. The condition c is constructed from the transcription of the target speaker. During training, the instruction takes the form

```
{Mix audio} Separate the sound of {transcription}.
```
(16)

The meanings of the individual components in the instruction are the same as those described in Section H.1.

Model Adaptation, Training Strategy, and Trainable Parameters.

Since this task requires the model to understand the linguistic content of the input audio, we follow the adaptation used for our base editor and replace the original T5 text encoder in SAM Audio with MOSS-Audio-4B-Thinking [34]. We connect MOSS to the SAM Audio DiT through the same Meta Query [21] mechanism. The overall architecture is identical to that shown in Figure 37, except that the DiT backbone of our base editor is replaced by the SAM Audio DiT.

Table 19 Trainable parameters.

Module	Params.
SAM Audio LoRA	127.40M
Queries	0.13M
Connector	65.54M
Projector	18.09M
Total	**211.17M**

We similarly adapt the SAM Audio DiT with LoRA, using rank 256, alpha 512, and dropout 0.05. We also directly compare standard Flow Matching training with training that incorporates the early-trajectory emphasis principle from the beginning. During training, we use the same set of trainable components as described in Section H.1. The numbers of trainable parameters for different modules are summarized in Table 19.

Training Configuration. We initialize the model from the publicly released SAM Audio Small [28] checkpoint and train it on this task for 20 epochs using either Standard FM or the early-emphasis training strategy. Both settings use AdamW with a weight decay of 0.01, a peak learning rate of 5×10^{-5}, 1,000 optimizer-step warmup followed by cosine decay, a batch size of 56, and gradient clipping with a maximum norm of 1.0.

Since several trainable components, including the DiT LoRA, Connector, and Projector, are newly initialized, this setting does not fully satisfy the original assumption of STEER-Emphasis that the model already possesses an initial task capability. We therefore adopt the same mixed sampling strategy as in the experiment with our base editor: while most training states follow the STEER-Emphasis t sampling, a small proportion are sampled using the standard logit-normal distribution to retain coverage of the full generation trajectory. Specifically, Standard FM uses the standard logit-normal timestep sampling strategy $t \sim \mathrm{sigmoid}\big(\mathcal{N}(0,1)\big)$. For early-emphasis training, we use

$$t \sim \begin{cases} \mathcal{U}(0, 0.03), & \text{with probability } 0.8, \\ \mathrm{sigmoid}\big(\mathcal{N}(0,1)\big), & \text{with probability } 0.2. \end{cases}$$

As described in Appendix E.1, SAM Audio adopts the opposite time parameterization from our base editor, where $t = 0$ corresponds to noise and $t = 1$ corresponds to clean audio. Accordingly, 80% of the training states are concentrated in the high-noise early-trajectory region, while the remaining 20% retain standard logit-normal sampling to preserve coverage of the full generation trajectory. The training objective remains the standard Flow Matching objective in both settings.

Training Dataset. We use the same PromptTSE [27] dataset as in the experiment with our base editor. In our TSS setting, the target-speaker audio and residual speech are used as the supervision targets for the two output streams.

Evaluation Metric. We use CLAP Audio–Audio similarity for TSS (CLAP-$\mathrm{AA}_{\mathrm{TSS}}$) to evaluate the separation results of SAM Audio. Unlike CLAP-$\mathrm{AA}_{\mathrm{TSR}}$, which only measures the similarity between the generated residual audio and the ground-truth residual audio, CLAP-$\mathrm{AA}_{\mathrm{TSS}}$ jointly evaluates both separated outputs while penalizing similarity to the incorrect source. Let $E_a(\cdot)$ denote the CLAP [5] audio encoder, and define the cosine similarity between two audio signals as

$$C(a, b) = \frac{E_a(a)^\top E_a(b)}{\|E_a(a)\|_2 \, \|E_a(b)\|_2}.$$

For sample i, the model produces the predicted target speech $\hat{s}_{\mathrm{target},i}$ and predicted residual speech $\hat{s}_{\mathrm{residual},i}$, with corresponding ground-truth signals $s_{\mathrm{target},i}$ and $s_{\mathrm{residual},i}$. We define

$$\begin{aligned} \text{CLAP-AA}_{\mathrm{TSS},i} = \frac{1}{2}\Big[& C(\hat{s}_{\mathrm{target},i}, s_{\mathrm{target},i}) - C(\hat{s}_{\mathrm{target},i}, s_{\mathrm{residual},i}) \\ & + C(\hat{s}_{\mathrm{residual},i}, s_{\mathrm{residual},i}) - C(\hat{s}_{\mathrm{residual},i}, s_{\mathrm{target},i}) \Big]. \end{aligned}$$

The final score is averaged over all N evaluation samples:

$$\text{CLAP-AA}_{\text{TSS}} = \frac{1}{N}\sum_{i=1}^{N}\text{CLAP-AA}_{\text{TSS},i}\,.$$

$\text{CLAP-AA}_{\text{TSS}}$ therefore encourages the predicted target speech to be more similar to the target ground truth than to the residual speech, while simultaneously encouraging the predicted residual speech to be more similar to the residual ground truth than to the target speech. A higher score indicates better separation of the target and non-target speakers.

Experimental Results.
Figure 39 shows the validation $\text{CLAP-AA}_{\text{TSS}}$ of Standard FM and early trajectory emphasis training over 20 training epochs. Compared with Standard FM, early trajectory emphasis training learns this fine-grained speaker separation task substantially more effectively. This result further demonstrates the importance of emphasizing the training on critical early-trajectory states for fine-grained selective control. Together with the results for our base editor, it further shows that this early-trajectory emphasis training principle remains effective across different model architectures and task formulations.

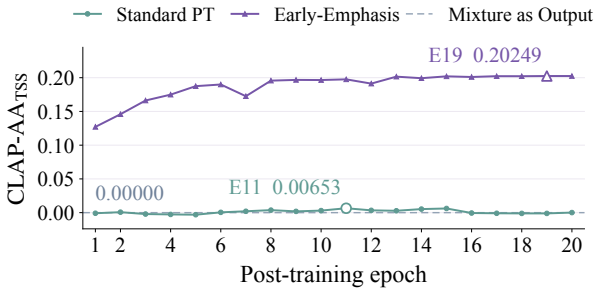


Figure 39 Validation $\text{CLAP-AA}_{\text{TSS}}$ during training on the TSS task with SAM Audio. The horizontal line denotes the $\text{CLAP-AA}_{\text{TSS}}$ of the mixture audio.

The horizontal line denotes the $\text{CLAP-AA}_{\text{TSS}}$ obtained by directly using the unseparated mixed audio as the model output. Standard FM achieves a CLAP-AA score near zero, whereas early-trajectory emphasis substantially improves performance on this fine-grained selective-control task.

I Human Evaluation and Website Design

I.1 Human Annotation Procedure and Criteria

As reported in Sections 5.4, and Appendices F.2, we conduct human evaluations of our base editor and its post-trained variants on the Complex Audio Editing Test Set and of task-adapted versions of our base editor and SAM Audio on the TSR and TSS tasks.

For the Complex Audio Editing Data, we largely follow the error-type definitions in Appendix C.2. The only difference is that the input audio may contain multiple sources. Therefore, a sample is categorized as Unintended Non-Target Edit whenever the target source undergoes the intended edit while at least one non-target source is also modified. For the TSR and TSS tasks, annotators are asked to determine whether the output audio satisfies the task requirement and to label each result as either successful or unsuccessful. For both annotation tasks, annotators are allowed to abstain when a sample is too complex to judge reliably. Such samples are excluded from the final statistics. During annotation, annotators are not informed which model produced each audio sample.

I.2 Website Design

We develop two web-based annotation interfaces for human evaluation. For the Complex Audio Editing Test Set, annotators are shown the editing instruction and the reference, target, and output audio, and classify each result according to the success and selective-error categories (Figure 40). For the TSR and TSS tasks, we use a simplified interface in which annotators judge each output as Success, Failure, or Unable to Judge (Figure 41). Model identities are hidden in both interfaces.

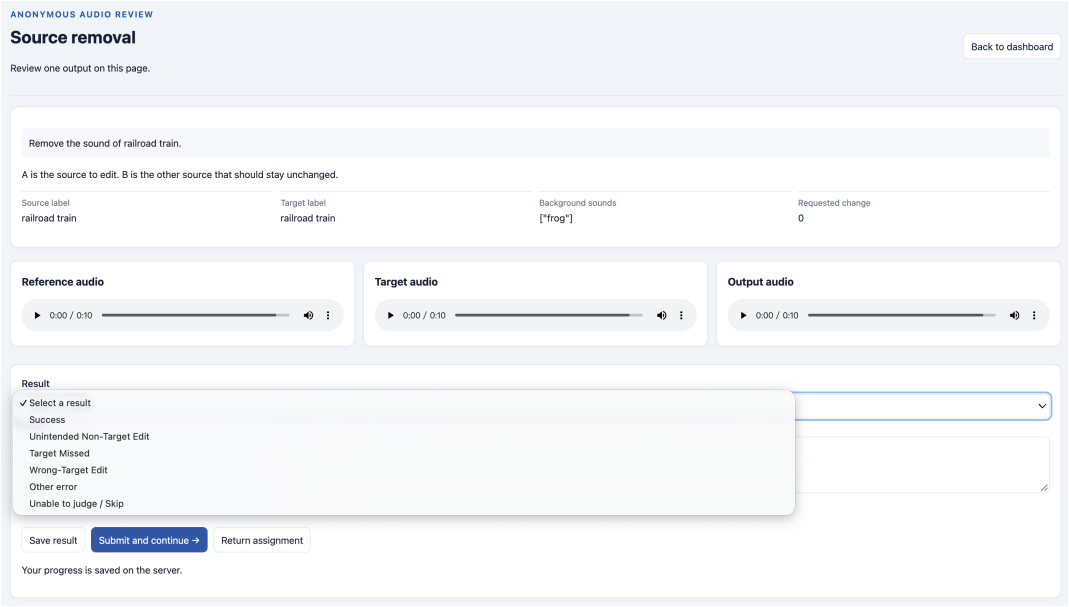


Figure 40 Human evaluation interface for the Complex Audio Editing Test Set.

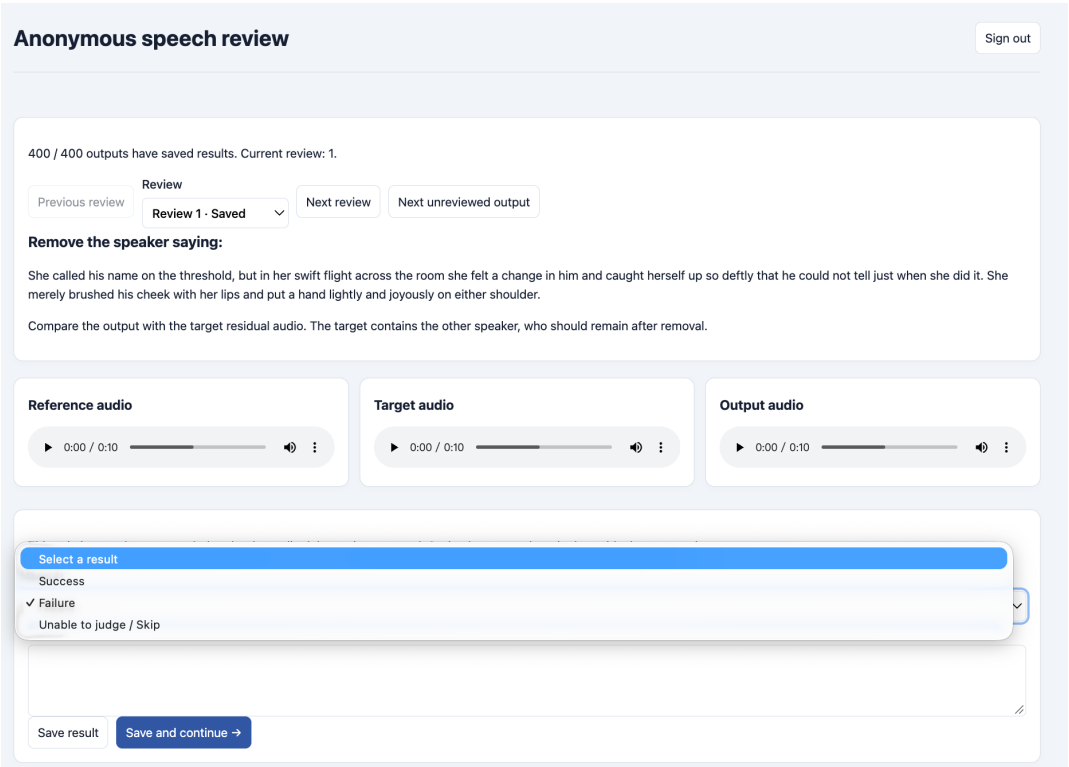


Figure 41 Human evaluation interface for the TSR and TSS tasks.